

Summary of Changes: **BSIM-SOI_4.7.0**

BSIM Group, IIT Kanpur, UC Berkeley

List of developers: https://www.bsim.berkeley.edu/models/bsimsoi/auth_soi_a/

A. Summary of Enhancements:

1. **2017 enh.4:** Added OP information is making overlap capacitance contribution twice (Synopsys)
2. **2018 enh.1:** Adding contact resistance RSC and RDC (CADENCE & ADI)
3. **2018 enh.2:** Regarding temperature parameters trs and trd (CADENCE & ADI)
4. **2018 enh.3:** Removing superfluous variable in VA (ADI)
5. **2018 enh.4:** Removal of Redundant warnings – SHMOD (ADI)
6. **2018 enh.5:** Default value of MtrlMod constants (ADI)
7. **2018 enh.6:** Change in parameters range (ADI)
8. **2018 enh.7:** Warning: parameter in denominator and allowed to zero (ADI)
9. **2018 enh.8:** Enhancement: change from model parameter to model switch (ADI)
10. **2018 enh.9:** Correction in warning and fatal errors (ADI)
11. **2018 enh.10:** Vertical gate resistance in BSIMSOI (GF)
12. **2019 enh.1:** Consistency between parameters in BSIM-SOI manual and code (ADI)
13. **2019 enh.2:** Node collapse when Rd < min value (ADI)
14. **2019 enh.3:** GDS Operating point definition (ADI)
15. **2019 enh.4:** GMIN implementation (GMIN WG)
16. **2019 enh.5:** Check RSC_t and RDC_t are positive and bias-independent temperature dependent (ADI)
17. **2020 enh.1:** Correct the flicker noise implementation as per IEEE paper “Flicker Noise Formulations in Compact Models.” (ADI)
18. **2020 enh.2:** replace \$finish with \$error (ADI)
19. **2020 enh.3:** Removal of unused parameters (ADI)
20. **2020 enh.4:** Rs and Rd are superfluous for RDSMOD=2 (ADI)
21. **2020 enh.5:** Limiting the effective NSUB value to positive (UCB)
22. **2020 enh.6:** Replace “\$finish” with “\$error” (ADI)
23. **2020 enh.7:** GMIN declaration error (ADI)
24. **2020 enh.8:** Bias dependency of “B4SOIk1ox” (ADI)
25. **2021 enh.1:** IDSOMULTMOD enhancement for local layout Effects. (GF)
26. **2021 enh.2:** Mobility and S/D Resistance Enhancement for Back Gate Bias. (Cadence)
27. **2021 enh.3:** Temperature dependence in GIDL and gate current. (GF)
28. **2022 enh.1:** Removing repetition of model parameters. (ADI)
29. **2022 enh.2:** Modification of pin declaration in QA tests (ADI)
30. **2022 enh.3:** Modification of font-case for parameters (ADI)
31. **2022 enh.4:** Bias value update for some QA tests. (ADI)
32. **2022 enh.5:** Bias sweep doesn’t converge to final bias sweep (Simulator specific). (ADI, Silvaco)
33. **2022 enh.6:** Testcase name modification for QA test. (ADI)
34. **2023 enh.1:** MULT_I / MULT_Q / MULT_FN implementation. (GF)
35. **2023 enh.2:** Improved Indentation (ADI)
36. **2023 enh.3:** Make parameters case consistent (ADI)
37. **2024 enh.1:** Out of the total 1044 parameters in the model parameter file, no value was specified for 234 parameters (only the default value was tested for 68 parameters). (ADI)
38. **2024 enh.2:** Add QA test for all MOD switches (ADI)
39. **2024 enh.3:** Instance parameters are read as model parameters for simulators that enforce instance vs model parameters (ADI)
40. **2025 enh.1:** QA tests update. (ADI/Cadence)
41. **2025 enh.2:** TNOIMOD =3 not enabled by default in the code (ADI)

B. Summary of Enhancements:

1. **2018 bug1:** Stress effect in vfbzb – Coding issue (ADI)
 2. **2018 bug2:** Self heating module – Updating K1ox and K2ox (ADI)
 3. **2018 bug3:** BSIMSOI model fitting discrepancy at negative VE– No VA code update required. (Infineon)
 4. **2018 bug4:** Nonphysical behavior of Sid and Sig in TNOIMOD=1 (GF)
 5. **2019 bug1:** Convergence issue at low temperature (CADENCE)
 6. **2019 bug2:** QA test Idvd_diode_st., value 0.25 appears twice in bias List (GF)
 7. **2020 bug1:** A bulk CVfactor correction – pow (0,0) (ADI)
 8. **2020 bug2:** Diode leakage components & Negative binning values (Cadence & GF)
 9. **2020 bug3:** Operating variable: Correction in Macro definition (ADI)
 10. **2020 bug4:** Parameter's unit update (ADI)
 11. **2020 bug5:** Bug in RDSMOD implementation (ADI)
 12. **2020 bug6:** Technical manual update: Tnom correction (UCB)
 13. **2021 bug1:** Negative value of CDD. (CADENCE)
 14. **2021 bug3:** Vampyre Warning “possible division by zero for expression $1/(1-ASD)$ ”. (ADI)
 15. **2021 bug3:** Incorrect value of CGB. (GF)
 16. **2022 bug1:** Allow “minr=0”. (ADI)
 17. **2023 bug1:** Revert IDS0MULT parameter (GF)
 18. **2023 bug2:** Improved MULT implementation (GF)
 19. **2023 bug3:** Updating fundamental constants to pass QA tests
 20. **2024 bug1:** The range of FDMOD is "no bound"; should be restricted to 0 or 1. (ADI)
 21. **2024 bug2:** Possible division by zero for few parameters. (ADI)
 22. **2024 bug5:** Correct RDSMOD = 1 implementation in code (UCB/IITK)
 23. **2024 bug6:** Correct source/drain resistance equations for RDSMOD = 1 in technical manual (UCB/IITK)
 24. **2024 bug7:** Issues with TRm1 replacing (TempRatio -1) (UCB/IITK)
 25. **2024 bug8:** Code cleaning – B4SOItnom is replaced with Tnom, and (Tratio -1) is replaced with TRm1 (UCB/IITK)
 26. **2025 bug1:** Vampyre Warning “possible division by zero for parameter “SCALEN”. (IITK/UCB)
 27. **2025 bug2:** VA path correction in qaSpec file and result folder path correction in QA Makefile. (ADI)
-

Description of Enhancements:

1. 2017 enh.4: Added OP information is making overlap capacitance contribution twice (Synopsys)

Solution:

```
Operating point information added in BSIM-SOI 4.6.0 & BSIM-SOI 4.6.1
// [Nov 2016] : adding overlap charges (qgso + qgdo) to qgate
// [Qov depends on V(gm,si) for RGATEMOD=3, else on V(gi,si)]
qgi = qgate;
qov = qgso + qgdo;
qgate = qgi + qov;
```

Part 1 of the code

As below, later on at the stamping side it makes overlap capacitance contribution twice

```
I(gi,b) <+ B4SOItype * ddt(qgate);
if (RGATEMOD == 3) begin
    I(gm,di) <+ B4SOItype * ddt(qgdo); I(gm,si) <+ B4SOItype * ddt(qgso);
    I(gm, e) <+ ddt(V(gm,e) * pParam_B4SOIcgeo);
end else begin
    I(gi,di) <+ B4SOItype * ddt(qgdo); I(gi,si) <+ B4SOItype * ddt(qgso);
    I(gi, e) <+ ddt(V(gi,e) * pParam_B4SOIcgeo);
end
```

Part 2 of the code

Response: Part 1 of the code is moved below part 2 of the code and OP output

2. 2018 enh.1: Adding contact resistance RSC and RDC (Synopsys)

Solution:

Parameter definition and initialization:

```
`MPRCz( RSC,      0,      "ohm",      "Source contact resistance")
`MPRCz( RDC,      0,      "ohm",      "Drain contact resistance")
```

Bias independent contact resistance RSC and RDC irrespective of RDSMOD:

```
if (RDSMOD == 1) begin /* 2013.enh5, 2015.enh1 */ /* Updated in v4.6 */
    Rds = 0.0;
    /* Rs(V) */
    T0 = vgs - pParam_B4SOIvfbds;
    T1 = sqrt(T0 * T0 + 1.0e-4);
    vgs_eff = 0.5 * (T0 + T1);

    T0 = 1.0 + pParam_B4SOIprwg * vgs_eff;
    T1 = -pParam_B4SOIprwb * vbs;
    T2 = 1.0 / T0 + T1;
    T3 = T2 + sqrt(T2 * T2 + 0.01);

    T4 = rs0 * 0.5;
    Rs = rswmin + T3 * T4 + B4SOIsourceResistance + RSC0;

    /* Rd(V) */
    T0 = vgd - pParam_B4SOIvfbds;
    T1 = sqrt(T0 * T0 + 1.0e-4);
    vgd_eff = 0.5 * (T0 + T1);

    T0 = 1.0 + pParam_B4SOIprwg * vgd_eff;
    T1 = -pParam_B4SOIprwb * vbd;

    T2 = 1.0 / T0 + T1;
    T3 = T2 + sqrt(T2 * T2 + 0.01);
    T4 = rd0 * 0.5;
    Rd = rdwmin + T3 * T4 + B4SOIdrainResistance + RDC0;

end else begin
    Rs = B4SOIsourceResistance + RSC0;
    Rd = B4SOIdrainResistance + RDC0;

end

/* v4.6 */
if (RDSMOD == 2) begin
    Rds = B4SOIdrainResistance + Rds + B4SOIsourceResistance + RDC0 + RSC0;
end
B4SOIrrds = Rds / NF; /* Fix v4.31 bug in Thermal noise for TNOIMOD=0 */
```

NOTE: In the standard code these RSC0 and RDC0 are later replaced with RSC_t and RDC_t to address 2019enh.2 and 2019enh.5.

3. 2018 enh.2: Regarding temperature parameters trs and trd (CADENCE & ADI)

Solution:

Parameter definition and initialization

```
`MPRCz( TRS ,      0,      "ohm",      " TEMP Dependence of Source contact resistance" )
`MPRCz( TRD ,      0,      "ohm",      " TEMP Dependence of Drain contact resistance" )
```

```
//Temp dependent RSC RDC
RSC0 = RSC + TRS * (TempRatio - 1.0);
RDC0 = RDC + TRD * (TempRatio - 1.0);
/* v4.6 */
```

4. 2018 enh.3: Removing superfluous variable in VA (ADI)

Solution: Following variables are removed:

B4SOItsi	B4SOILint	B4SOILl	B4SOIbetaGB2	B4SOIcgsI
B4SOIbigc	B4SOIsaref	B4SOIckappa	B4SOIua	B4SOIunitLengthGate SidewallUctCapD
B4SOIlgcMod	B4SOIweffcot	B4SOIntrecf	B4SOIa0	B4SOIrdsw
B4SOIsoiMod	B4SOItempeot	B4SOIub1	B4SOIcigbcp2	B4SOIrhato
B4SOIbeta1	B4SOIIsd	B4SOImoinFD	B4SOIvecb	B4SOIvsdfb
B4SOIBGBC2P	B4SOIketas	B4SOILwl	B4SOInrecr0	B4SOIdvt1
B4SOIute	B4SOIw	B4SOIWlc	B4SOIalphaGB1	B4SOIvbci
B4SOIkgsI	B4SOIBGB2	B4SOIngcon	B4SOIvbsa	B4SOIleasub
B4SOIucs	B4SOIngate	B4SOIk3	B4SOIipe0	B4SOIvfb
B4SOIisdif	B4SOIvgb1	B4SOIldif0	B4SOIebjtii	B4SOIud1
B4SOInoff	B4SOItnoia	B4SOIxrcrg2	B4SOIdvtp3	B4SOIepsrox
B4SOIni0sub	B4SOILwn	B4SOILwlc	B4SOIk1w2	B4SOIndioded
B4SOIuc	B4SOItoxqm	B4SOIdtoxcv	B4SOIcdscd	B4SOIkj
B4SOIetabcv	B4SOIkt1	B4SOIfdMod	B4SOIcapMod	B4SOIk2b
B4SOIdvtp1	B4SOIWint	B4SOIkvt0	B4SOIbdos	B4SOIk1b
B4SOIvtun0	B4SOIcit	B4SOIllodvth	B4SOIrgateMod	B4SOIvtun0d
B4SOIk1	B4SOIrswwin	B4SOIados	B4SOIegisl	B4SOIu0
B4SOIEGGDEP	B4SOIrbodv	B4SOIvth0	B4SOIminv	B4SOIndiode
B4SOItpbswg	B4SOIacde	B4SOIEGGBCP2	B4SOILlc	B4SOIntox
B4SOIdvt1w	B4SOIwlodku0	B4SOIdlcig	B4SOIistun	B4SOIetsi
B4SOIaf	B4SOIWln	B4SOIrth0	B4SOIagisl	B4SOInrecf0d
B4SOIrdc	B4SOIpdits	B4SOIdrainPerimeter	B4SOIdelta	B4SOInofffd
B4SOItnoic	B4SOIxt	B4SOIa2	B4SOIdvt2w	B4SOIdrout
B4SOIcdsc	B4SOIdk2b	B4SOImobMod	B4SOIigbMod	B4SOIb1
B4SOIsourceArea	B4SOIwlodvth	B4SOIdvtp2	B4SOIAGB2	B4SOIxrecd
B4SOIahlid	B4SOIIsa	B4SOIdvbd1	B4SOIxbjt	B4SOIsteta0cv
B4SOItox	B4SOIaebcp	B4SOIdifd	B4SOIeu	B4SOIsteta0
B4SOIvsdth	B4SOIdvt0	B4SOIrdwmin	B4SOIbinUnit	B4SOInseg
B4SOIxrec	B4SOIdelvt	B4SOIsii1	B4SOIsbref	B4SOIpcIm
B4SOIvbx	B4SOIcigsd	B4SOIpdibl1	B4SOIepsrgate	B4SOIeta0
B4SOItoxref	B4SOIbodySquares	B4SOIpditsl	B4SOIags	B4SOIdibjt

B4SOIprt	B4SOIbetaGB1	B4SOIbodyJctGateSideD GradingCoeff	B4SOIvbs0fd	B4SOIoxideTrapDensi tyA
B4SOIxtun	B4SOIntund	B4SOIrnoib	B4SOIvddeot	B4SOIcth0
B4SOIgamma1	B4SOIclc	B4SOIfprout	B4SOInigc	B4SOIvofffd
B4SOIdvtp4	B4SOILw	B4SOIabjtii	B4SOIAGBC2N	B4SOIxtund
B4SOIepsrsub	B4SOIntrecre	B4SOIrbbsb	B4SOIpigcd	B4SOIcscdesw
B4SOIsii2	B4SOIvrec0	B4SOIagidl	B4SOIebg	B4SOIua1
B4SOIdvt2	B4SOInpeak	B4SOIvsce	B4SOIbodyJctGateSide SGradingCoeff	B4SOInfactor
B4SOIlefeot	B4SOIgamma2	B4SOItpbswgd	B4SOIdvbd0	B4SOInbc
B4SOIucste	B4SOIdlc	B4SOInoff2	B4SOIalphaGB2	B4SOIcgidl
B4SOItii	B4SOIa1	B4SOIwr	B4SOIrs	B4SOIphig
B4SOIAGBC2P	B4SOIeot	B4SOIpdiblb	B4SOIk1	B4SOIisbjt
B4SOIaigsd	B4SOIat	B4SOIsiid	B4SOIWwl	B4SOIxcrg1
B4SOIrbdb	B4SOIunitLengthGateSidewallJct CapS	B4SOIvoffcv	B4SOIoxideTrapDensit yC	B4SOIw0flk
B4SOIprwg	B4SOIbigbcp2	B4SOItype	B4SOIdwg	B4SOIlii
B4SOIbeta0	B4SOIntun	B4SOIgbmin	B4SOIxgl	B4SOIllodku0
B4SOIWwlc	B4SOILln	B4SOIllodk2	B4SOInbjt	B4SOIrbbodyMod
B4SOIsourcePeri meter	B4SOIbgidl	B4SOItbgasub	B4SOIntnoi	B4SOIub
B4SOIvdsatii0	B4SOIpoxedge	B4SOIsheetResistance	B4SOItcjswgd	B4SOIrbsh
B4SOImtrlMod	B4SOIeta0cv	B4SOIVTM00	B4SOIrgidl	B4SOIdlcb
B4SOIcdsbs	B4SOIfbjtii	B4SOIpditsd	B4SOIk2	B4SOIdrainSquares
B4SOIcle	B4SOIvevb	B4SOIaigbcp2	B4SOIcigc	B4SOIWwn
B4SOIaely	B4SOIwth0	B4SOIkt1l	B4SOIsourceSquares	B4SOIud
B4SOIWwc	B4SOIcgisl	B4SOIvoxh	B4SOIkgidl	B4SOIprwb
B4SOItnoib	B4SOIbeta2	B4SOIBGB1	B4SOIvsat	B4SOIndif
B4SOIbgisl	B4SOItcswg	B4SOIkt2	B4SOIdvtp0	B4SOIxdif
B4SOIesatii	B4SOImoin	B4SOIegidl	B4SOIagbcp2	B4SOIalpha0
B4SOInrecre0d	B4SOIetab	B4SOIb0	B4SOIfgidl	B4SOIrbbody
B4SOImbjtii	B4SOIw0	B4SOIrbshg	B4SOIrmoc	B4SOIdwb
B4SOInsub	B4SOIAGB1	B4SOIvabjt	B4SOIBGBC2N	B4SOIvgb2
B4SOIk3b	B4SOIdeltavox	B4SOItt	B4SOIbjtoff	B4SOIdvtp0w
B4SOIrdMod	B4SOIrbbody	B4SOIsb	B4SOIvoff	B4SOIcdscb
B4SOInf	B4SOIxgw	B4SOIsii0	B4SOItoxm	B4SOIliiMod
B4SOIpdcbp	B4SOIdlbg	B4SOIdwbc	B4SOIpvag	B4SOIrbjt0
B4SOIfgisl	B4SOInrecre0	B4SOIku0	B4SOIbg0sub	B4SOIk1w1
B4SOIpsbcp	B4SOInsd	B4SOIvgstcvMod	B4SOIahli	B4SOIcgidl
B4SOIllodeta0cv	B4SOIisrec	B4SOIWl	B4SOIvrec0d	B4SOItbox
B4SOIbigsd	B4SOIvbm	B4SOIllodeta0	B4SOIbf	B4SOIoxideTrapDensi tyB
B4SOIem	B4SOIdsub	B4SOIkf	B4SOIgidlMod	B4SOIstk2
B4SOIdrec	B4SOIrdw	B4SOILn	B4SOIlpeb	B4SOIrgisl
B4SOInoif	B4SOIcbjtii	B4SOILwc	B4SOIxpart	B4SOItku0
B4SOIagbcpd	B4SOIpdibl2	B4SOIaigc	B4SOIiddif	B4SOIuc1
B4SOItbgsub	B4SOIrnoia	B4SOIvbs0pd	B4SOIas	B4SOIdrainArea
B4SOIWw	B4SOIdelvt0	B4SOIrc	B4SOIdtun	B4SOIl
B4SOIef	B4SOIagbcp	B4SOIEGGBCP2	B4SOIEGGDEP	B4SOIAGB1
B4SOIBGB1	B4SOIAGB2	B4SOIBGB2	B4SOIAGBC2N	B4SOIAGBC2P
B4SOIBGBC2N	B4SOIBGBC2P	B4SOIVTM00		

5. 2018 enh.4: Removal of Redundant warnings – SHMOD (ADI)

Solution: Following redundant warnings are removed:

```
4821 if ($param_given(K1) || $param_given(K2)) begin // {
4822     if (!$param_given(K1)) begin
4823         $strobe( "Warning: k1 should be specified with k2.");
4824         pParam_B4SOIk1 = 0.53;
4825     end
4826     if (!$param_given(K2)) begin
4827         $strobe( "Warning: k2 should be specified with k1.");
4828         pParam_B4SOIk2 = -0.0186;
4829     end
4830     if ($param_given(XT))
4831         $strobe( "Warning: xt is ignored because k1 or k2 is given.");
4832     if ($param_given(VBX))
4833         $strobe( "Warning: vbx is ignored because k1 or k2 is given.");
4834     if ($param_given(VBM))
4835         $strobe( "Warning: vbm is ignored because k1 or k2 is given.");
4836     if ($param_given(GAMMA1))
4837         $strobe( "Warning: gamma1 is ignored because k1 or k2 is given.");
4838     if ($param_given(GAMMA2))
4839         $strobe( "Warning: gamma2 is ignored because k1 or k2 is given.");
4840 end else begin // {}
```

The updated code is as follows:

```
4821 if ($param_given(K1) || $param_given(K2)) begin // {
4822     if (!$param_given(K1)) begin
4823         pParam_B4SOIk1 = 0.53;
4824     end
4825     if (!$param_given(K2)) begin
4826         pParam_B4SOIk2 = -0.0186;
4827     end
4828 end else begin // {}
```

6. 2018 enh.5: Default value of MtrlMod constants (ADI)

Solution: Constants' values were changed in BSIMSOI 4.6.0 and BSIMSOI 4.5.1 as follows:

`MPRnb(AGBC2N	,3.4254e-7	,	"	, "NMOS 'A' for tunneling current model")
`MPRnb(AGBC2P	,4.9723e-7	,	"	, "PMOS 'A' for tunneling current model")
`MPRnb(BGBC2N	,1.1665e12	,	"	, "NMOS 'B' for tunneling current model")
`MPRnb(BGBC2P	,7.4567e11	,	"	, "PMOS 'B' for tunneling current model")

Now, values for these constants are corrected as follows:

`MPRnb(AGBC2N	,3.42537e-7	,	"	, "NMOS 'A' for tunneling current model")
`MPRnb(AGBC2P	,4.97232e-7	,	"	, "PMOS 'A' for tunneling current model")
`MPRnb(BGBC2N	,1.16645e12	,	"	, "NMOS 'B' for tunneling current model")
`MPRnb(BGBC2P	,7.45669e11	,	"	, "PMOS 'B' for tunneling current model")

7. 2018 enh.6: Change in parameters range (ADI)

Solution: Below listed parameters cannot be negative. Hence, parameter range is changed from no bound to positive value as follows:

`MPRnb(VSAT	,8.0e4	, "m/s"	, "Saturation velocity at tnom")
`MPRnb(NSUB	,6.0e16	, "cm^-3"	, "Substrate doping concentration with polarity")
`MPRnb(NCH	,1.7e17	, "cm^-3"	, "Channel doping concentration")
`MPRnb(NSD	,1.0e20	, "cm^-3"	, "S/D doping concentration")
`MPRnb(CJSWG	,1e-10	, "F/m^2"	, "Source(gate side) sidewall junction capacitance per unit width")
`MPRnb(CJSWGD	,CJSWG	, "F/m^2"	, "Drain (gate side) sidewall junction capacitance per unit width")

Change

`MPRnb -> `MPRoz

Changed to 0→∞
(EXCLUDED ZERO)

`MPRnb(NGATE	,0.0	, "cm^-3"	, "Poly-gate doping concentration")
`MPRnb(CGSO	,0.0	, "F/m"	, "Gate-source overlap capacitance per width")
`MPRnb(CGDO	,0.0	, "F/m"	, "Gate-drain overlap capacitance per width")
`MPRnb(RSH	,0.0	, "ohm"	, "Source-drain sheet resistance")
`MPRnb(RDSW	,100	, ""	, "Source-drain resistance per width")
`MPRnb(RSW	,50	, "ohm(um)^(WR)"	, "Source resistance per width /* v4.0")
`MPRnb(RDW	,50	, "ohm(um)^(WR)"	, "Drain resistance per width /* v4.0")
`MPRnb(RSC	,0	, "ohm"	, "Source contact resistance")
`MPRnb(RDC	,0	, "ohm"	, "Drain contact resistance")
`MPRnb(RSWMIN	,0.0	, "ohm(um)^(WR)"	, "Source resistance per width at high Vg")
`MPRnb(RDWMIN	,0.0	, "ohm(um)^(WR)"	, "Drain resistance per width at high Vg")
`MPRnb(CF	,0.0	, "F/m"	, "Fringe capacitance parameter")

Change

`MPRnb -> `MPRcz

Changed to 0→∞

`MPRnb(NTUN	,10.0	, ""	, "Reverse tunneling non-ideality factor")
`MPRnb(NTUND	,NTUN	, ""	, "Reverse tunneling non-ideality factor")
`MPRnb(NRECFO	,2.0	, ""	, "Recombination non-ideality factor at forward bias")
`MPRnb(NRECFOD	,NRECFO	, ""	, "Recombination non-ideality factor at forward bias")
`MPRnb(NRECRO	,10.0	, ""	, "Recombination non-ideality factor at reversed bias")
`MPRnb(NRECROD	,NRECRO	, ""	, "Recombination non-ideality factor at reversed bias")
`MPRnb(ISBJT	,1e-6	, "A/m^2"	, "BJT injection saturation current")
`MPRnb(IDBJT	,ISBJT	, "A/m^2"	, "BJT injection saturation current")
`MPRnb(ISDIF	,0.0	, "A/m^2"	, "Body to source/drain injection saturation current")
`MPRnb(IDDIF	,ISDIF	, "A/m^2"	, "Body to source/drain injection saturation current")
`MPRnb(ISREC	,1e-5	, "A/m^2"	, "Recombination in depletion saturation current")
`MPRnb(IDREC	,ISREC	, "A/m^2"	, "Recombination in depletion saturation current")
`MPRnb(ISTUN	,0.0	, "A/m^2"	, "Reverse tunneling saturation current")
`MPRnb(IDTUN	,ISTUN	, "A/m^2"	, "Reverse tunneling saturation current")
`MPRnb(LN	,2e-6	, "m"	, "Electron/hole diffusion length")
`MPRnb(TT	,1e-12	, "s"	, "Diffusion capacitance transit time coefficient")

Change

`MPRnb -> `MPRcz

Changed to 0→∞

`IPRnb(NRB	,1.0	, ""	, "Number of squares in body")
`IPRnb(NBC	,0.0	, ""	, "Number of body contact isolation edge")
`IPRnb(PDBCP	,0.0	, "m"	, "Perimeter length for bc parasitics at drain side")
`IPRnb(PSBCP	,0.0	, "m"	, "Perimeter length for bc parasitics at source side")
`IPRnb(AGBCP	,0.0	, "m^2"	, "Gate to body overlap area for bc parasitics")
`IPRnb(AGBCP2	,0.0	, "m^2"	, "Parasitic Gate to body overlap area for bc parasitics")
`IPRnb(AGBCPD	,AGBCP	, "m^2"	, "Gate to body overlap area for bc parasitics in DC")
`IPRnb(AEBCP	,0.0	, "m^2"	, "Substrate to body overlap area for bc parasitics")

Change `IPRnb -> `IPRcz

Changed to 0→∞

8. 2018 enh.7: Warning: parameter in denominator and allowed to be zero (ADI)

Issue: Compiler gives warning while parameter appears in denominator and allowed to be zero.

```

▪ `IPRnb( NSEG      ,1      ,""      , "Number segments for width partitioning" )

pParam_B4SOLwdiod = pParam_B4SOLweff / NSEG + PDBCP;
pParam_B4SOLwdios = pParam_B4SOLweff / NSEG + PSBCP;

```

▪ Similar Issue with below listed parameters,

```

`MPRnb( NGCON      ,1.0      ,""      ,1      ,inf      , "Number of gate contacts" )
`MPRnb( ASD        ,0.3      ,""      , "Source/drain bottom diffusion smoothing parameter" )
`MPRnb( LEFFEOT     ,1.0      , "um"   , "Effective length for extraction of EOT" )
`MPRcz( EPSRSUB     ,11.7     ,""      , "Dielectric constant of substrate relative to vacuum" )
`MPRnb( VEB        ,0.075     ,""      , "Vaux parameter for valence-band electron tunnelling" )
`MPRnb( VECB        ,0.026     ,""      , "Vaux parameter for conduction-band electron tunneling" )
`MPRnb( TSI         ,1e-7      , "m"     , "Silicon-on-insulator thickness in meters" )
`MPRnb( XJ          ,TSI       , "m"     , "Junction Depth" )
`MPRnb( ETSI        ,1e-7      , "m"     , "Effective Silicon-on-insulator thickness in meters" )
`MPRnb( TBOX        ,3e-7      , "m"     , "Back gate oxide thickness in meters" )

```

```

`IPRnb( NSEG      ,1      ,""      , "Number segments for width partitioning" )
`MPRnb( NGCON      ,1.0      ,""      ,1      ,inf      , "Number of gate contacts" )

```

Solution: Parameters range is changed from no bound range to positive (restricted to zero)

```

`IPRco( NSEG      ,1      ,""      ,1      ,inf      , "Number segments for width partitioning" )
`IPRco( NGCON      ,1.0      ,""      ,1      ,inf      , "Number of gate contacts" )

`MPRnb( ASD        ,0.3      ,""      , "Source/drain bottom diffusion smoothing parameter" )
`MPRnb( LEFFEOT     ,1.0      , "um"   , "Effective length for extraction of EOT" )
`MPRcz( EPSRSUB     ,11.7     ,""      , "Dielectric constant of substrate relative to vacuum" )
`MPRnb( VEB        ,0.075     ,""      , "Vaux parameter for valence-band electron tunnelling" )
`MPRnb( VECB        ,0.026     ,""      , "Vaux parameter for conduction-band electron tunneling" )
`MPRnb( TSI         ,1e-7      , "m"     , "Silicon-on-insulator thickness in meters" )
`MPRnb( XJ          ,TSI       , "m"     , "Junction Depth" )
`MPRnb( ETSI        ,1e-7      , "m"     , "Effective Silicon-on-insulator thickness in meters" )
`MPRnb( TBOX        ,3e-7      , "m"     , "Back gate oxide thickness in meters" )
`MPRnb( CKAPPA      ,0.6      , "F/m"   , "New C-V model parameter" )

```

Parameters range is changed from

`MPRnb to → `MPRoz

Changed to 0→∞
(EXCLUDED ZERO)

9. 2018 enh.8: Change from model parameter to model switch (ADI)

Issue: Compiler gives warning while parameter appears in denominator and allowed to be zero. PARAMCHK and NODECHK should be model switch rather than model parameters.

```

`MPInb( PARAMCHK    ,0      ,""      , "Model parameter checking selector" )
`MPInb( NODECHK     ,1      ,""      , "NODE checking flag" ) /* Added for Node Checking in v4.6 */

```

Solution: PARAMCHK and NODECHK are changed to model switch as follows:

```

`MPIsw( NODECHK     ,1      ,""      , "NODE checking flag" )
`MPIsw( PARAMCHK    ,0      ,""      , "Model parameter checking selector" )

```


10. 2018 enh.9: Correction in warning and fatal errors (ADI)

Issue: Wrong warning messages and wrong parameter ranges.

Solution: Corrected the warnings and their value as follows:

if (NTUN < 0.0) \$strobe("Warning: Ntuns = %g is negative.", NTUN);	→	Changed to Ntun
if (VREC0 < 0.0) \$strobe("Warning: Vrec0s = %g is negative.", VREC0);	→	Changed to Vrec0
if (VTUN0 < 0.0) \$strobe("Warning: Vtun0s = %g is negative.", VTUN0);	→	Changed to Vtun0
if (AHLI < 0.0) \$strobe("Warning: Ahlis = %g is negative.", AHLI);	→	Changed to Ahli
if (AGISL < 0.0) \$strobe("Warning: Agidl = %g is negative.", AGIDL);	→	Changed to AGISL

☐ if (pParam_B4SOlpdibl1 < 0.0) \$strobe("Warning: Pdibl1 = %g is negative.", pParam_B4SOlpdibl1); ☐ if (pParam_B4SOlpdibl2 < 0.0) \$strobe("Warning: Pdibl2 = %g is negative.", pParam_B4SOlpdibl2);	→	Changed to Pdiblc1
	→	Changed to Pdiblc2

pParam_B4SOlpdibl1 = PDIBLC1 + LPDIBLC1* Inv_L + WPDIBLC1* Inv_W + PPDIBLC1* Inv_LW;
pParam_B4SOlpdibl2 = PDIBLC2 + LPDIBLC2* Inv_L + WPDIBLC2* Inv_W + PPDIBLC2* Inv_LW;

Fatal error issue	☐ if (TOXREF < 0.0) begin \$strobe("Warning: Toxref = %g is negative.", TOXREF); Fatal_Flag = 1; end Response: ☐ if (TOXREF < 0.0) begin \$strobe("Fatal: Toxref = %g is negative.", TOXREF); Fatal_Flag = 1; end
--------------------------	--

11. 2018 enh.10: Vertical gate resistance in BSIMSOI (GF)

Solution:

Defined new parameter:

`MPRCz(RVER ,0.0 , "ohm" , "Vertical Gate sheet resistance")

Existing Model:

$$R_g = \frac{rshg \left(\frac{W_f}{3 \cdot ngcon} + xgw \right)}{ngcon \cdot (L - xgl) \cdot N_f} \quad (1)$$

Updated Model:

Present BSIM Model	Additional part
$R_g = \frac{rshg \left(\frac{W_f}{3 \cdot ngcon} + xgw \right)}{ngcon \cdot (L - xgl) \cdot N_f}$	$+ \frac{rint + rvpoly}{L \cdot W_f \cdot N_f}$
Rsili	Rint
New parameter RVER is defined Reference: [B. Dormieu et. al, TED, 2013]	

12. 2019 enh.1: Consistency between parameters in BSIM-SOI manual and code (ADI)

Solution:

Old Equation in Technical Manual:

$$K_{stress_u0} = \left(1 + \frac{L_{KU0}}{(L_{drawn} + XL)^{LLODKU0}} + \frac{WKU0}{(W_{drawn} + XW + WLOD)^{WLODKU0}} \right) + \frac{PKU0}{(L_{drawn} + XL)^{LLODKU0} \cdot (W_{drawn} + XW + WLOD)^{WLODKU0}} \cdot \left(1 + TKU0 \cdot \left(\frac{Temperature}{TNOM} - 1 \right) \right)$$

Updated Equation in Technical Manual:

$$K_{stress_{uo}} = 1 + \frac{L_{KU0}}{(L_{drawn})^{LLODKU0}} + \frac{WKU0}{(W_{drawn} + WLOD)^{WLODKU0}} + \frac{PKU0}{(L_{drawn})^{LLODKU0} \cdot (W_{drawn} + WLOD)^{WLODKU0}} \left(1 + TKU0 \cdot \left(\frac{Temperature}{TNOM} - 1 \right) \right)$$

13. 2019 enh.2: Node collapse when Rd < min value (ADI)

Solution:

- Node collapsing Rd < Rmin
- Implementation of MinR and Node collapsing for RDSMOD=0.

```
//Temp dependent RSC RDC
RSC0 = RSC + TRS * (TempRatio - 1.0);
RDC0 = RDC + TRD * (TempRatio - 1.0);
```

```
Rd = rdwmin + T3 * T4 + B4SOIdrainResistance + RDC0 ;
end else begin
Rs = B4SOIsourceResistance + RSC0 ;
Rd = B4SOIdrainResistance + RDC0;
```

If TRD/TRS is negative, then RDC0/RSC0 may become negative. Then Rd/Rs may turn out to be negative. Check is only for "B4SOIdrainResistance". No check exists for Rd/Rs.

Fix:

`MPRoz(minr, \$simparam("minr", 1.0e-3) ,"ohm" ,"minr is the value below which the simulator expects elimination of source/drain resistance, and it will improve simulation efficiency without significantly altering the results.")

Updated Code:

For RDSMOD = 0

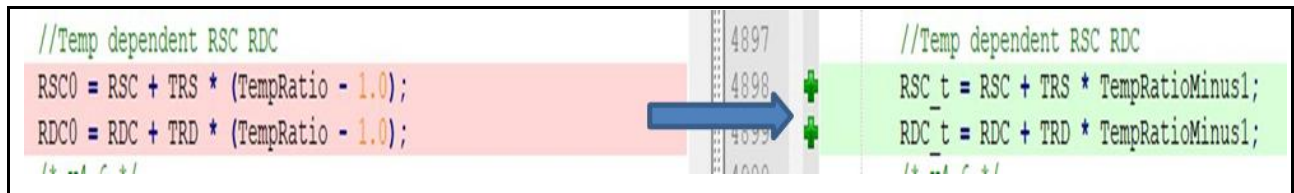
```
if (RDSMOD == 0) begin
  if ((B4SOIsourceResistance + RSC) > MINR) begin
    Rs = B4SOIsourceResistance + RSC_t ;
    if (Rs < MINR) begin
      Rs = MINR;
    end
  end else begin
    Rs = 0;
  end

  if ((B4SOIdrainResistance + RSC) > MINR) begin
    Rd = B4SOIdrainResistance + RDC_t;
    if (Rd < MINR) begin
      Rd = MINR;
    end
  end else begin
    Rd = 0;
  end
end
```

This is check for clamping the value of Rs to MINR

Node collapsing check

Nomenclature also changed as follows:



For RDSMOD = 1

Clamp Rd/Rs values to MINR=1e-3

```

end if (RDSMOD == 1) begin /* 2013.enh5, 2015.enh1 */ /* Updated in v4.6 */
  Rds = 0.0;
  /* Rs(V) */
  T0 = vgs - pParam_B4SOIvfb;
  T1 = sqrt(T0 * T0 + 1.0e-4);
  vgs_eff = 0.5 * (T0 + T1);

  T0 = 1.0 + pParam_B4SOIprwg * vgs_eff;
  T1 = -pParam_B4SOIprwb * vbs;
  T2 = 1.0 / T0 + T1;
  T3 = T2 + sqrt(T2 * T2 + 0.01);

  T4 = rs0 * 0.5;
  Rs = rswmin + T3 * T4 + B4SOIsourceResistance + RSC_t;
  if (Rs < MINR) begin
    Rs = MINR;
  end
end

```

Note: Similar changes are made for drain resistance (Rd).

More relevant changes in code:

BSIM-SOI v4.6.1:

```

/* process source/drain series resistance */
B4SOIdrainResistance = B4SOIsheetResistance * B4SOIdrainSquares;
if (B4SOIdrainResistance <= 1.0e-3) /* 2013.enh5, 2015.enh1 */ /* v4.6 */
  B4SOIdrainResistance = 1.0e-3;

B4SOIsourceResistance = B4SOIsheetResistance * B4SOIsourceSquares;

if (B4SOIsourceResistance <= 1.0e-3) /* 2013.enh5, 2015.enh1 */ /* v4.6 */
  B4SOIsourceResistance = 1.0e-3;

```

BSIM-SOI v4.7.0 Beta2:

```

/* process source/drain series resistance */
B4SOIdrainResistance = RSH * NRD;
if (RDSMOD == 1 && B4SOIdrainResistance < MINR) begin
  B4SOIdrainResistance = MINR;
end
B4SOIsourceResistance = RSH * NRS;
if (RDSMOD == 1 && B4SOIsourceResistance < MINR) begin
  B4SOIsourceResistance = MINR;
end

```

14. 2019 enh.3: GDS Operating point definition (ADI)

Issue:

Issue :

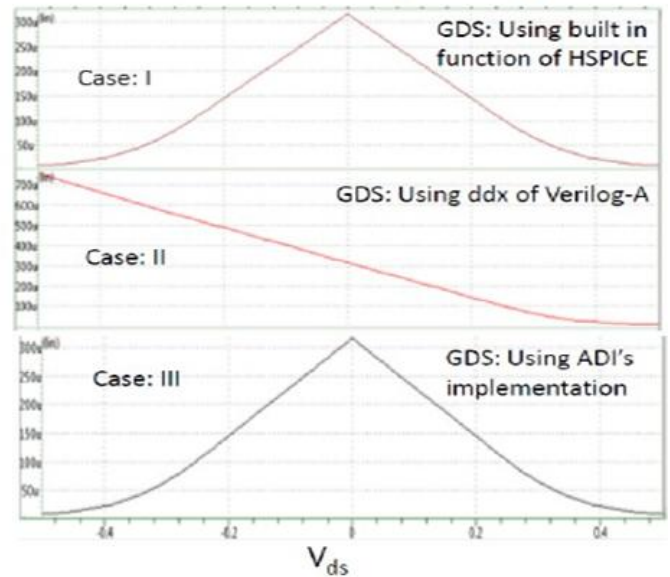
- GDS using ddx of Verilog A (Case II) is not symmetric. It should be symmetric as shown in case I.

BSIM-BULK107.0.0_Beta3

```
GDS = ddx(IDS, V(di));
```

BSIM-BULK107.0.0_Beta4

```
if (sigvds > 0)begin
GDS = ddx(IDS, V(di));
end
else begin
GDS = ddx(-IDS, V(si));
end
```

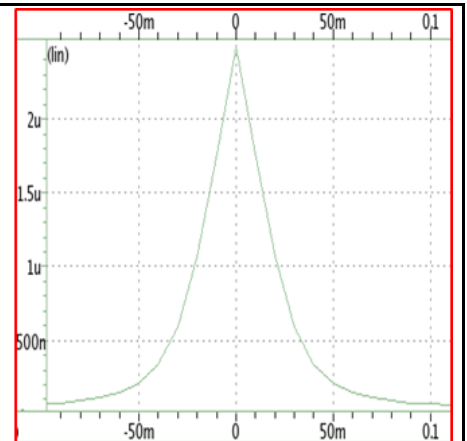


Gummel Symmetry Test circuit is used for biasing scheme in all cases.

Solution: No code update required. Present implementation of GDS Op in BSIM-SOI is same as in BSIM-BULK.

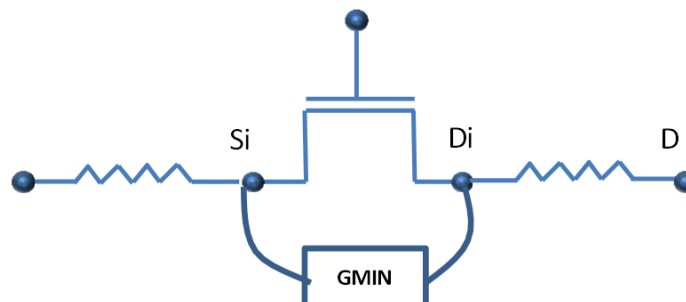
```
if (B4SOImode > 0)
B4SOIgds = TYPE * ddx(Ids, V(di));
else
B4SOIgds = TYPE * ddx(Ids, V(si));

// operating-point information
IDS = TYPE * Ids;
GM = B4SOIgm;
GDS = B4SOIgds;
```



15. 2019 enh.4: GMIN implementation (GMIN WG)

Solution:



Updated Code:

```
if (B4SOImode > 0) begin
    I(di,si) <+ TYPE * (Ids + Ic) + ($simparam("gmin", 0) * V(di, si));
    I(di,b) <+ TYPE * Iii;
    B4SOIigidl = TYPE * Igidl;
    B4SOIigisl = TYPE * Igisl;
    B4SOIlgcd = TYPE * Igcd;
    B4SOIlgcs = TYPE * Igcs;
    B4SOIqdrn = TYPE * qdrn;
    B4SOIqsrc = TYPE * qsrc;
end else begin
    I(si,di) <+ TYPE * (Ids - Ic) + ($simparam("gmin", 0) * V(si, di));
    I(si,b) <+ TYPE * Iii;
    B4SOIigisl = TYPE * Igidl;
    B4SOIigidl = TYPE * Igisl;
    B4SOIlgcs = TYPE * Igcd;
    B4SOIlgcd = TYPE * Igcs;
    B4SOIqsrc = TYPE * qdrn;
    B4SOIqdrn = TYPE * qsrc;
end
```

16. 2019 enh.5: Check RSC_t and RDC_t are positive and bias independent temperature dependent (ADI)

Issue: RSC0 and RDC0 must be positive.

Solution: Temp dependent RSC and RDC (RDSMOD=0):

RSC = RSC + TRS * (TempRatio - 1.0); RDC0 = RDC + TRD * (TempRatio - 1.0);	Old Code
---	-----------------

RSC_t = RSC + TRS * TempRatioMinus1; RDC_t = RDC + TRD * TempRatioMinus1;	New Code
--	-----------------

NOTE: TempRatioMinus1 is bias independent temperature dependence.

17. 2020 enh.1: Correction of the flicker noise implementation as per IEEE paper “Flicker Noise Formulations in Compact Models” (ADI)

Solution: Instantaneous noise power approach → Need to supply sign of modulation function based on following paper:

G. J. Coram, et. al., “Flicker Noise Formulations in Compact Models,” Transaction on Computer-Aided Design of integrated Circuits and Systems, 13 January 2020.

Added part in the code:

if (B4SOImode < 0) begin flickerNoiseContrib = - flickerNoiseContrib; end

Calling flicker noise:

I(di,si) <+ flicker_noise(flickerNoiseContrib, EF, "flicker");
--

18. 2020 enh.2: Replace \$finish with \$error (ADI)

Solution: \$finish is replaced with \$error at every instance as follows:

Old Code:

```

if (pParam_B4SOIleff <= 0.0) begin
    $strobe("Effective channel length <= 0");
    $finish(0);
end

```

New Code:

```

if (pParam_B4SOIleff <= 0.0) begin
    $error("Effective channel length <= 0");
end

```

19. 2020 enh.3: Removal of unused parameters (ADI)**Issue:** Following parameters are never used in the code:

```

Parameter 'VBSUSR' was never used
Parameter 'DEBUG' was never used
Parameter 'OFF' was never used
Parameter 'TOXP' was never used

```

Solution: These parameters have been removed from the code.**20. 2020 enh.4:** Rs and Rd are superfluous for RDSMOD=2 (ADI)**Issue:** Rs and Rd are superfluous for RDSMOD=2 in BSIM-SOI 470 Beta1:

```

if (RDSMOD == 2) begin          /* 2013.enh5, 2015.enh1 */ /* v4.6 */
    Rs = 0 + RSC0;
    Rd = 0 + RDC0;
end

```

Solution: The superfluous Rs and Rd are removed in BSIM-SOI v 4.7.0 Beta2. Following is the snapshot from the updated code:

```

if (RDSMOD == 0) begin
    .
    .
    .
end if (RDSMOD == 1) begin
    .
    .
    .
end else begin // RDMOD == 2
    Rs = 0;
    Rd = 0;
end

```

21. 2020 enh.5: Limiting the effective NSUB value to positive (UCB)**Issue:** Effective NSUB parameter evaluation:

```

pParam_B4SOInsub = NSUB + LNSUB * Inv_L + WNSUB * Inv_W + PNSUB * Inv_LW;

```

The **pParam_B4SOInsub** is used inside square root as follows:

```

pParam_B4SOIgamma2 = sqrt2qeps * sqrt(pParam_B4SOInsub) / B4SOIcox;

```

If **pParam_B4SOInsub** goes negative, it will give rise to issues. Currently, only NSUB is limited between 0 to inf. No limit on the binning parameters. Physically, effective substrate doping cannot be negative.

Solution: Included an error message following the evaluation of **pParam_B4SOInsub** as follows:

```
if (pParam_B4SOInsub < 0.0) begin
    $error("Effective substrate doping pParam_B4SOInsub cannot be negative");
end
```

22. 2020 enh.6: Replace “\$finish” with “\$error” (ADI)

Issue: Vampyre error: NOTICE in file bsimsoi.va, line 3505: \$error() preferred over \$finish().

Old Code (BSIM-SOI v 4.7.0 Beta2):

```
3504         if (Fatal_Flag)
3505             $finish(0);
```

Solution: We have replaced \$finish with \$error as follows:

```
if (Fatal_Flag)
    $error("Terminating due to previous fatal error.");
```

23. 2020 enh.7: GMIN declaration error (ADI)

Issue: WARNING in file bsimsoi.va, line 1275: Variable 'the_module.gmin' was never set and never used.

Solution: Removed the following gmin declaration from the code:

```
1273     real B4SOIIgd;
1274     real B4SOIIgs;
1275     real gmin;
```

24. 2020 enh.8: Bias dependency of “B4SOIk1ox” (ADI)

Issue: WARNING in file bsimsoi.va, line 5560: Bias-dependent '==': if.

```
if (pParam_B4SOIk1ox == 0.0) /* v4.0 */ begin
    Voxdepinv = 0.0;
end else begin
    .
    .
    .
```

K1OX is bias dependent and “if-else” may give rise to discontinuity.

Solution: Based on the discussion and agreement with work group, **we have decided not to make any changes in the code.** Only “pParam_B4SOIk1ox” has been renamed as “here_B4SOIk1ox” due to its bias-dependency.

25. 2020 enh.9: IDSOMULTMOD enhancement for local layout Effects (LLE) (GF)

Solution: To Introduce scaling of internal currents, source/drain resistances and mobility for LLE.

Updated implementation in BSIM-SOI_4.7.0_Beta4 is shown below:

A. Mobility scaling:

```
ueff = (u0temp + UBG2 *(ves -vfb)) / Denomi;
ueff = ueff * U0MULT;
B4SOIueff = ueff;
```

- Default value: U0MULT=IDS0MULT=1 is Backward compatible.

B. Drain current scaling:

```
T0 = 1.0 + T9;
Ids = Idl * T0 / NSEG;
Ids = Ids * IDS0MULT;
```

- Drain to source current is multiplied by IDS0MULT

C. External resistance scaling

```
if ( IDS0MULTMOD !=0 ) begin
  Rs = Rs / IDS0MULT;
  Rd = Rd / IDS0MULT;
end
```

- For scaling mode =1 or 2, drain/source resistances are divided by IDS0MULT.

D. Modification of various current components:

- All the second order current contributions to source and drain nodes are multiplied with IDS0MULT.
- Va code for updated implementation in BSIM-SOI_ 4.7.0_Beta4 is shown below:

```
if (B4SOImode > 0) begin
  I(di,si) <+ TYPE * (Ids + Ic) + ($simparam("gmin", 0) * V(di, si));
  I(di,b) <+ TYPE * Iii;
  if ( IDS0MULTMOD != 0 ) begin
    B4SOIigidl = TYPE * IDS0MULT * Igidl;
    B4SOIigisl = TYPE * IDS0MULT * Igisl;
    B4SOIlgcd = TYPE * IDS0MULT * Igcd;
    B4SOIlgcs = TYPE * IDS0MULT * Igcs;
  end else begin
    B4SOIigidl = TYPE * Igidl;
    B4SOIigisl = TYPE * Igisl;
    B4SOIlgcd = TYPE * Igcd;
    B4SOIlgcs = TYPE * Igcs;
  end
  B4SOIqdrn = TYPE * qdrn;
  B4SOIqsrc = TYPE * qsrc;
end else begin
  I(si,di) <+ TYPE * (Ids - Ic) + ($simparam("gmin", 0) * V(si, di));
  I(si,b) <+ TYPE * Iii;
  if ( IDS0MULTMOD != 0 ) begin
    B4SOIigisl = TYPE * IDS0MULT * Igidl;
    B4SOIigidl = TYPE * IDS0MULT * Igisl;
    B4SOIlgcs = TYPE * IDS0MULT * Igcd;
    B4SOIlgcd = TYPE * IDS0MULT * Igcs;
  end else begin
    B4SOIigisl = TYPE * Igidl;
```

```

        B4SOIigidl = TYPE * Igisl;
        B4SOIigcs = TYPE * Igcd;
        B4SOIigcd = TYPE * Igcs;
    end
    B4SOIqsrc = TYPE * qdrn;
    B4SOIqdrn = TYPE * qsrc;
end
if ( IDS0MULTMOD != 0 ) begin
    B4SOIigd = TYPE * IDS0MULT * Igd;
    B4SOIigs = TYPE * IDS0MULT * Igs;
end else begin
    B4SOIigd = TYPE * Igd;
    B4SOIigs = TYPE * Igs;
end
end

```

E. IDS0MULTMOD implementation for self-heating:

```

`ifdef PORT_CONNECTED
    if ((SHMOD == 1) && (RIH0 != 0.0)) begin /* 2014bug2 */ /* v4.6 */
        if (INODEOUT && !$port_connected(t)) begin
            if ($port_connected(b)) begin // last terminal is 'b'
                I(b) <+ -Ids * Vds + ddt(delTemp * pParam_B4SOIcth) + delTemp / pParam_B4SOIrtth;
            end else if ($port_connected(p)) begin // last terminal is 'p'
                I(p) <+ -Ids * Vds + ddt(delTemp * pParam_B4SOIcth) + delTemp / pParam_B4SOIrtth;
            end else begin // 4-terminal mode ('t' is internal)
                if (IDS0MULTMOD == 2) begin
                    Pwr(t) <+ -(Ids / IDS0MULT) * Vds + ddt(delTemp * pParam_B4SOIcth) + delTemp / pParam_B4SOIrtth;
                end else begin
                    Pwr(t) <+ -Ids * Vds + ddt(delTemp * pParam_B4SOIcth) + delTemp / pParam_B4SOIrtth;
                end
            end
        end else begin
            if (IDS0MULTMOD == 2) begin
                Pwr(t) <+ -(Ids / IDS0MULT) * Vds + ddt(delTemp * pParam_B4SOIcth) + delTemp / pParam_B4SOIrtth;
            end else begin
                Pwr(t) <+ -Ids * Vds + ddt(delTemp * pParam_B4SOIcth) + delTemp / pParam_B4SOIrtth;
            end
        end
    end else begin
        if (INODEOUT && !$port_connected(t)) begin
            if ($port_connected(b)) begin // last terminal is 'b'
                V(b) <+ 0;
            end else if ($port_connected(p)) begin // last terminal is 'p'
                V(p) <+ 0;
            end else begin // 4-terminal mode ('t' is internal)
                Temp(t) <+ 0;
            end
        end else begin
            Temp(t) <+ 0;
        end
    end
end
`else
    if ((SHMOD == 1) && (RIH0 != 0.0))
        if (IDS0MULTMOD == 2) begin
            Pwr(t) <+ -(Ids / IDS0MULT) * Vds + ddt(delTemp * pParam_B4SOIcth) + delTemp / pParam_B4SOIrtth;
        end else begin
            Pwr(t) <+ -Ids * Vds + ddt(delTemp * pParam_B4SOIcth) + delTemp / pParam_B4SOIrtth;
        end
    else
        Temp(t) <+ 0;
    end
`endif

```

Old Implementation:

`MPRcc(U0MULT	,1.0	,""	,0	,inf	,"Variability in carrier mobility")
----------------	------	-----	----	------	--------------------------------------

Modified Implementation:

`MPRcc(U0MULT	,1.0	,""	,0	,1	,"Variability in carrier mobility")
----------------	------	-----	----	----	--------------------------------------

26. 2021 enh.2: Mobility and S/D resistance enhancement for back gate bias (Cadence)

Solution: Parameters **UBG1** and **UBG2** for mobility model enhancement and **PRWE** for S/D resistance enhancement with back gate bias are added in BSIM-SOI_4.7.0_Beta4.

$$\mu_{eff} = \frac{\mu_0 + \text{UBG2} (V_{bGS} - V_{FBb})}{1 + (U_a + U_c V_{bseff} + \text{UBG1} (V_{bGS} - V_{FBb})) \left(\frac{V_{gsteff} + 2V_{th}}{T_{ox}} \right) + U_b \left(\frac{V_{gsteff} + 2V_{th}}{T_{ox}} \right)^2}$$

$$R_d = R_s = \frac{RDWMIN + RDW \left[-PRWB \cdot V_{bd} - \text{PRWE} (V_{bGS} - V_{FBb}) + \frac{1}{1 + PRWG (V_{gd} - V_{fbsd})} \right]}{(1e6 \cdot W_{eff})^{WR} \cdot NF}$$

Parameters to be added are encircled

Va code for updated implementation in BSIM-SOI_4.7.0_Beta4 is shown below:

1. Mobility implementation:

```
/* Mobility calculation */
if (MTRLMOD) begin
    T14 = 2.0 * TYPE * (PHIG - EASUB - 0.5 * Eg + 0.45);
    tox_mob = EOT * EPSRSUB / 3.9; /* Bug fix #4 Jun 09 implementing Eeff correctly */
    T15 = UBG1 * (ves - vfbb);
end else begin
    T14 = 0.0;
    tox_mob = TOX;
    /* Bug fix #4 Jun 09 implementing Eeff correctly */
    T15 = UBG1 * (ves - vfbb);
end
.
.
.
if (T5 >= -0.8) begin
    Denomi = 1.0 + T5;
end else begin
    /* Added to avoid the discontinuity problem caused by ua and ub */
    T9 = 1.0 / (7.0 + 10.0 * T5);
    Denomi = (0.6 + T5) * T9;
end
ueff = (u0temp + UBG2 * (ves - vfbb)) / Denomi;
ueff = ueff * U0MULT;
B4SOIueff = ueff;
```

2. S/D resistance implementation:

```
end else if (RDSMOD == 1) begin /* 2013.enh5, 2015.enh1 */ /* Updated in v4.6 */
.
.
/* Rd(V) */
T0 = vgd - pParam_B4SOIvfbsd;
T1 = sqrt(T0 * T0 + 1.0e-4);
vgd_eff = 0.5 * (T0 + T1);
T0 = 1.0 + pParam_B4SOIprwg * vgd_eff;
```



```

T1 = -pParam_B4SOIprwb * vbd;
T2 = 1.0 / T0 + T1 + pParam_B4SOIprwe * ( ves - vfb);
T3 = T2 + sqrt(T2 * T2 + 0.01);
T4 = rd0 * 0.5;
Rd = rdwmin + T3 * T4 + B4SOIdrainResistance + RDC_t ;
if (Rd < MINR) begin
    Rd = MINR;
end
end else begin // RDMOD == 2
    Rs = 0;
    Rd = 0;
end
end
if ( IDS0MULTMOD !=0 ) begin
    Rs = Rs / IDS0MULT;
    Rd = Rd / IDS0MULT;
end
end

```

3. Parameter PRWE is also made binnable:

```

pParam_B4SOIprwe = PRWE + LPRWE * Inv_L + WPRWE * Inv_W + PPRWE * Inv_LW;

```

27. 2021 enh.3: Temperature dependence in GIDL and gate currents (GF)

Solution: In BSIM-SOI_4.7.0_Beta4 release, the temperature dependency in GIDL and gate currents is added into model.

A. Temperature dependency in GIDL implementation:

$$\begin{aligned}
 I_{GIDL} &= AGIDL \cdot W_{diod} \cdot Nf \cdot \frac{V_{ds} - RGIDL \cdot V_{gse} - EGIDL + V_{fbsd}}{3 \cdot T_{oxe}} \\
 &\quad \times \exp\left(-\frac{3 \cdot T_{oxe} \cdot BGIDL}{V_{ds} - V_{gse} - EGIDL}\right) \cdot \exp\left(\frac{KGIDL}{V_{ds} - FGIDL}\right) \\
 I_{GISL} &= AGISL \cdot W_{dlos} \cdot Nf \cdot \frac{-V_{ds} - RGISL \cdot V_{gse} - EGISL + V_{fbsd}}{3 \cdot T_{oxe}} \\
 &\quad \times \exp\left(-\frac{3 \cdot T_{oxe} \cdot BGISL}{-V_{ds} - V_{gse} - EGISL}\right) \cdot \exp\left(\frac{KGISL}{-V_{bs} - FGISL}\right) \\
 \mathbf{BGIDL} &= \mathbf{BGIDL} + \mathbf{BGIDL1} * \left(\frac{DevTemp}{Tnom} - 1\right)
 \end{aligned}$$

VA code for updated implementation is shown below:

```

agidl = pParam_B4SOIagidl;
bgidl = pParam_B4SOIbgidl + pParam_B4SOIbgidl1 * TempRatioMinus1;
cgidl = pParam_B4SOIcgidl;
egidl = pParam_B4SOIegidl;
rgidl = pParam_B4SOIrgidl;
kgidl = pParam_B4SOIkgidl;
fgidl = pParam_B4SOIfgidl;
agisl = pParam_B4SOIagisl;

bgisl = pParam_B4SOIbgisl + pParam_B4SOIbgisl1 * TempRatioMinus1;
cgisl = pParam_B4SOIcgisl;
egisl = pParam_B4SOIegisl;

```

```

rgisl = pParam_B4SOIrgisl;
kgisl = pParam_B4SOIkgisl;
fgisl = pParam_B4SOIfgisl;

```

B. Temperature dependence in gate currents:

In inversion,

$$J_{gb} = A \frac{V_{gb} V_{aux}}{T_{ox}^2} \left(\frac{T_{oxref}}{T_{oxqm}} \right)^{N_{tox}} \exp \left(\frac{-B(\alpha_{gb1} - \beta_{gb1} |V_{ox}|) T_{ox}}{1 - |V_{ox}|/V_{gb1}} \right) * Igtemp$$

$$V_{aux} = V_{EVB} \ln \left(1 + \exp \left(\frac{|V_{ox}| - \Phi_g}{V_{EVB}} \right) \right)$$

$$A = \frac{q^3}{8\pi h \phi_b}$$

$$B = \frac{8\pi \sqrt{2m_{ox}} \phi_b^{3/2}}{3hq}$$

$$\phi_b = 4.2eV$$

$$m_{ox} = 0.3m_0$$

In accumulation,

$$J_{gb} = A \frac{V_{gb} V_{aux}}{T_{ox}^2} \left(\frac{T_{oxref}}{T_{oxqm}} \right)^{N_{tox}} \exp \left(\frac{-B(\alpha_{gb2} - \beta_{gb2} |V_{ox}|) T_{ox}}{1 - |V_{ox}|/V_{gb2}} \right) * Igtemp$$

$$V_{aux} = V_{ECB} V_t \ln \left(1 + \exp \left(-\frac{V_{gb} - V_{fb}}{V_{ECB}} \right) \right)$$

$$A = \frac{q^3}{8\pi h \phi_b}$$

$$B = \frac{8\pi \sqrt{2m_{ox}} \phi_b^{3/2}}{3hq}$$

$$\phi_b = 3.1eV$$

$$m_{ox} = 0.4m_0$$

Igtemp = $IGT * \left(\frac{DevTemp}{Tnom} \right)$ → Same is done for I_{gc} , I_{cs} , I_{cd} , I_{cs} and

$$\alpha_{gb1} = \alpha_{gb1} + \alpha_{gb1_t} * \left(\frac{DevTemp}{Tnom} - 1 \right)$$

$$\alpha_{gb2} = \alpha_{gb2} + \alpha_{gb2_t} * \left(\frac{DevTemp}{Tnom} - 1 \right)$$

C. Binned parameters in BSIM_SOI_4.7.0_Beta4 release

```

pParam_B4SOIxi = XJ + LXJ * Inv_L + WXJ * Inv_W + PXJ * Inv_LW;

pParam_B4SOIalphaGB1 = ALPHAGB1 + LALPHAGB1 * Inv_L + WALPHAGB1 * Inv_W + PALPHAGB1 * Inv_LW;

pParam_B4SOIalphaGB1_t = ALPHAGB1_t + LALPHAGB1_t * Inv_L + WALPHAGB1_t * Inv_W + PALPHAGB1_t * Inv_LW;

pParam_B4SOIalphaGB2 = ALPHAGB2 + LALPHAGB2 * Inv_L + WALPHAGB2 * Inv_W + PALPHAGB2 * Inv_LW;

```

```

igtemp = exp(pParam_B4SOIigt * ln(TempRatio));

pParam_B4SOIaigc = pParam_B4SOIaigc + pParam_B4SOIaigc1 * TempRatioMinus1;

pParam_B4SOIaigsd = pParam_B4SOIaigsd + pParam_B4SOIaigsd1 * TempRatioMinus1;

pParam_B4SOIalphaGB1 = pParam_B4SOIalphaGB1 + pParam_B4SOIalphaGB1_t * TempRatioMinus1;

```

```
pParam_B4SOIalphaGB2 = pParam_B4SOIalphaGB2 + pParam_B4SOIalphaGB2_t * TempRatioMinus1;
pParam_B4SOIaigbcp2 = pParam_B4SOIaigbcp2 + pParam_B4SOIaigbcp2_t * TempRatioMinus1;
```

28. 2022 enh.1: Removing repetition of model parameters (ADI)

Issue: Parameter SOIMOD/MOBSMOD defined more than once.

WARNING: parameter SOIMOD defined more than once, last value specified will be used
WARNING: parameter MOBSMOD defined more than once, last value specified will be used

These parameters are listed twice, in a file and directly:

modelParameters	parameters/nmosParameters
modelParameters	SOIMOD=0 MOBSMOD=1

test	01_DC_Sweep1
biases	V(s)=0 V(b)=0
biasList	V(g)=-1.6,0.4,1.6
biasSweep	V(d)=0.1,1.5,0.1
outputs	I(d)
instanceParameters	W=0.2e-6 L=0.04e-6
modelParameters	parameters/nmosParameters
modelParameters	SOIMOD=0 MOBSMOD=1

test	02_DC_Sweep2
biases	V(s)=0 V(b)=0
biasList	V(g)=-1.6,0.4,1.6
biasSweep	V(d)=0.1,1.5,0.1
outputs	I(d)
instanceParameters	W=0.2e-6 L=0.04e-6
modelParameters	parameters/nmosParameters
modelParameters	SOIMOD=1 MOBSMOD=2

This also has SOIMOD, MOBSMOD

Model parameter SOIMOD/MOBSMOD defined twice.

Here we are modifying SOIMOD, MOBSMOD values define in parameter file

Solution: Parameters to be modified in testcases will be defined separately for each test (& declaration is removed from the nmosParameters).

29. 2022 enh.2: Modification of pin declarations in QA tests (ADI)

Issue: Some simulators (which connects by pin name) fail tests because as per specification they will connect to pin b (optional) but will have no connection to e (required one).

4 of the 6 simulators I have appear to fail every test.

One of the simulators connects by pin name, so this specification:

pins d g s b

will connect to pin "b" (which is optional) but will have no connection to "e" (which is required).

Solution: Pin "b" is changed to pin "e."

30. 2022 enh.3: Modification of font-case for parameters (ADI)

Issue: Some tests use uppercase parameter name and some lowercase.

test	02_DC_Sweep2
biases	V(s)=0 V(b)=0
biasList	V(g)=-1.6,0.4,1.6
biasSweep	V(d)=0.1,1.5,0.1
outputs	I(d)
instanceParameters	W=0.2e-6 L=0.04e-6
modelParameters	parameters/nmosParameters
modelParameters	SOIMOD=1 MOBSMOD=2

Uppercase names

test	Idvg_core_1w
biases	V(s)=0 V(b)=0
biasList	V(d)=0.05,1.0
biasSweep	V(g)=0.25,1.2,0.1
outputs	I(d) I(g) I(b) I(s)
instanceParameters	W=10.0e-6 L=1e-6
modelParameters	parameters/nmosParameters
modelParameters	igbmod=0 igcmmod=0 gidlmod=0

lowercase names

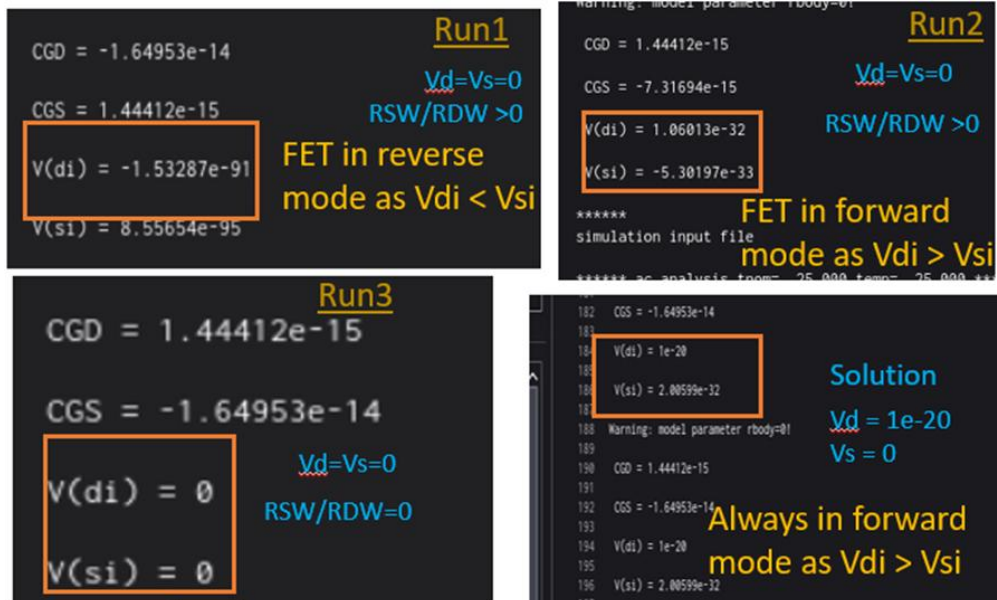
Solution: All parameter names are changed to uppercase.

31. 2022 enh.4: Bias value updation for some QA tests (ADI)

Issue: For some tests bias values used is 0.00000000000000000001, why not zero or 1e-20.

```
test      acvg_core_sw
biases    V(s)=0 V(b)=0
biasList  V(d)=0.00000000000000000001,1.0
```

For Bias value of zero:



- Even at $V_s=V_d=0$, V_{si} and V_{di} shows non-zero values due to source and drain resistances.
- And often, due to simulator specific numerical round-off, the values of V_{di} and V_{si} are not consistent and sometimes $V_{di} < V_{si}$ which forces the device into reverse mode of operation even at $V_d=V_s=0$.
- In reverse mode, the Source and Drain terminal gets swapped Cgs and Cgd values gets interchanged.
- As in simulations, $C_{gs} \neq C_{gd}$ (very minute difference always exists), so the mismatch appears in QA test.

Solution: Bias value updated to 1e-20.

32. 2022 enh.5: Bias sweep doesn't converge to final bias sweep (simulator specific) (ADI, SILVACO)

Issue: For most starting with "Idvg_core_": simulator used failed the standard variant, but passes Flip_N and has only a slight issue with m.

```
***** Checking test (SIMULATOR): Idvg_core_sw

variant: standard    (compared to: reference) FAIL    (probably from some simulation failure)
variant: Flip_N      (compared to: standard) MATCH    (within specified tolerances)
variant: m           (compared to: standard) DIFFER   (max rel error is 0.0293%)

The problem is in the biasSweep:

test      Idvg_core_sw
biases    V(s)=0 V(b)=0
biasList  V(d)=0.05,1.0
biasSweep V(g)=0.25,1.2,0.1

Starting at 0.25 in steps of 0.1 we will get 1.25 not 1.2 ,Hspice coverages to final
point irrespective of step size but, seems to fail in few simulators.
```

Solution: For these tests bias sweep is fixed to: biasSweep v(g)=0.25,1.15,0.1, so that bias sweep will converge to the final sweep point .

33. 2022 enh.6: Test name modification for QA test (ADI, SILVACO)

Issue:

```
1) the first 10 tests are numbered:

test      01_DC_Sweep1

test      02_DC_Sweep2

...

Can you we get that applied also to the remaining tests?

test      ldvg_core_sw

test      ldvg_core_lw
```

Solution: Test names modified as per the request.

34. 2023 enh.1: MULT_I / MULT_Q / MULT_FN implementation (GF)

Solution: New convenience parameters MULT_I, MULT_Q and MULT_FN are introduced. These are not intended to be physical, but introduced as a simple way to modify current, capacitance and noise. Parameters IDS0MULTMOD and IDS0MULT have been removed as MULT_I parameter does the same job. Moreover, MULT_I / MULT_Q / MULT_FN implementation is done in alignment with the CMC recommended guideline. The changes made in the code by this implementation are summarized by the table shown below.

	After MULT_I / MULT_Q / MULT_FN Implementation
Current	Scaled by MULT_I parameter
Charge	Scaled by MULT_Q parameter
Self-heating power	No change
White noise	Scaled by MULT_I parameter
Flicker noise	Scaled by MULT_FN parameter

35. 2023 enh.2: Improved Indentation (ADI)

Solution: Indentation in BSIM-SOI_4.7.0_Beta6 from BSIM-SOI_4.7.0_Beta5 is improved as follows:

BSIM-SOI_4.7.0_Beta5:

```
if ((SHMOD == 1) && (RTH0 != 0.0))
  if (IDS0MULTMOD == 2) begin
    Pwr(t) <+ -(Ids / IDS0MULT) * Vds + ddt(delTemp * pParam_B4SOIcth) + delTemp / pParam_B4SOIrh;
  end else begin
    Pwr(t) <+ -Ids * Vds + ddt(delTemp * pParam_B4SOIcth) + delTemp / pParam_B4SOIrh;
  end
else
  Temp(t) <+ 0;
```


BSIM-SOI_4.7.0_Beta6:

```
if ((SHMOD == 1) && (RTH0 != 0.0)) begin
  if (IDSOMULTMOD == 2) begin
    Pwr(t) <+ -(Ids / IDSOMULT) * Vds + ddt(delTemp * pParam_B4SOIcth) + delTemp / pParam_B4SOIrrh;
  end else begin
    Pwr(t) <+ -Ids * Vds + ddt(delTemp * pParam_B4SOIcth) + delTemp / pParam_B4SOIrrh;
  end
end else begin
  Temp(t) <+ 0;
end
```

36. 2023 enh.3: Make parameters case consistent (ADI)

Issue:

This is because you have some parameters with "_t" rather than "_T". I don't think you should have mixed-case parameters.

```
`MPRnb( PALPHAGB1 ,0.0 ,"" , "Cross-term dependence of alphagb1" )
`MPRnb( PALPHAGB1_t ,0.0 ,"" , "Cross-term dependence of alphagb1_t" )
`MPRnb( PBETAGB1 ,0.0 ,"" , "Cross-term dependence of betagb1" )
`MPRnb( PALPHAGB2 ,0.0 ,"" , "Cross-term dependence of alphagb2" )
`MPRnb( PALPHAGB2_t ,0.0 ,"" , "Cross-term dependence of alphagb2_t" )
`MPRnb( PBETAGB2 ,0.0 ,"" , "Cross-term dependence of betagb2" )
`MPRnb( PAIGBCP2 ,0.0 ,"" , "Cross-term dependence of aigbcp2" )
`MPRnb( PAIGBCP2_t ,0.0 ,"" , "Cross-term dependence of aigbcp2_t" )
```

Solution: All the letters in parameters' definition are now updated in Upper case as follows:

```
`MPRnb( PALPHAGB1 ,0.0 ,"" , "Cross-term dependence of alphagb1" )
`MPRnb( PALPHAGB1_T ,0.0 ,"" , "Cross-term dependence of alphagb1_t" )
`MPRnb( PBETAGB1 ,0.0 ,"" , "Cross-term dependence of betagb1" )
`MPRnb( PALPHAGB2 ,0.0 ,"" , "Cross-term dependence of alphagb2" )
`MPRnb( PALPHAGB2_T ,0.0 ,"" , "Cross-term dependence of alphagb2_t" )
`MPRnb( PBETAGB2 ,0.0 ,"" , "Cross-term dependence of betagb2" )
`MPRnb( PAIGBCP2 ,0.0 ,"" , "Cross-term dependence of aigbcp2" )
`MPRnb( PAIGBCP2_T ,0.0 ,"" , "Cross-term dependence of aigbcp2_t" )
```

37. 2024 enh.1: Out of the total 1044 parameters in the model parameter file, no value was specified for 234 parameters (only the default value was tested for 68 parameters) (ADI)

Solution: Added tests for SOIMOD = 2 , MOBMOD = 4 and TNOIMOD = 3. Added the specified 234 parameters to model parameter file.

```
test      03A_DC_Sweep3
biases    V(s)=0 V(e)=0
biasList  V(g)=-1.6,0.4,1.6
biasSweep V(d)=0.1,1.5,0.1
outputs   I(d)
instanceParameters W=0.2e-6 L=0.04e-6
modelParameters parameters/nmosParameters
modelParameters SOIMOD=2 MOBMOD=3 IGBMOD=0 IGCMOD=0 GIDLMOD=0 FNOIMOD=0 TNOIMOD=2 CAPMOD=3

test      03B_DC_Sweep3
biases    V(s)=0 V(e)=0
biasList  V(g)=-1.6,0.4,1.6
biasSweep V(d)=0.1,1.5,0.1
outputs   I(d)
instanceParameters W=0.2e-6 L=0.04e-6
modelParameters parameters/nmosParameters
modelParameters SOIMOD=2 MOBMOD=4 IGBMOD=0 IGCMOD=0 GIDLMOD=0 FNOIMOD=0 TNOIMOD=2 CAPMOD=3

test      03C_DC_Sweep3
biases    V(s)=0 V(e)=0
biasList  V(g)=-1.6,0.4,1.6
biasSweep V(d)=0.1,1.5,0.1
outputs   I(d)
instanceParameters W=0.2e-6 L=0.04e-6
modelParameters parameters/nmosParameters
modelParameters SOIMOD=1 MOBMOD=3 IGBMOD=0 IGCMOD=0 GIDLMOD=0 FNOIMOD=0 TNOIMOD=3 CAPMOD=3
```

38. 2024 enh.2: Add QA test for all MOD switches (ADI)

Solution: In the BSIM-SOI 4.7.0_beta8 QA package, we have added QA tests for both NMOS and PMOS so that all the MOD switches are tested.

39. 2024 enh.3: Instance parameters are read as model parameters for simulators that enforce instance vs model parameters (ADI)

Solution: Instance parameters are removed from the parameter folder in the BSIM-SOI_4.7.0_beta8 and added as instance parameters in the **qaSpec** file.

40. 2025 enh.1: QA tests update (ADI, CADENCE)

Solution: Added 6 new tests for TNOIMOD = 0,1, and 3 for both NMOS and PMOS. We renamed all the tests, corrected some parameter duplication in tests, and added a test for TNOIMOD = 0,1 and 3.

41. 2025 enh.2: TNOIMOD = 3 not enabled by default in the code (ADI)

Issue: In BSIM-SOI_4.7.0_Beta9 TNOIMOD = 3 is inside an `ifdef statement and the corresponding `define statement is commented out. To activate TNOIMOD = 3 we need to uncomment "`define _TNOIMOD3_" in the code.

Solution: Removed "`ifdef " for TNOIMOD = 3 and enabled it to default in the code similar to TNOIMOD = 0, 1 and 2. The "`define _TNOIMOD3_" and "`ifdef _TNOIMOD3_" statements are removed from the code.

BSIM-SOI_4.7.0_Beta9:

```

: begin
//Correlated Thermal Noise by Navid, July 2013
#ifdef _TNOIMOD3_

eta = 1.0 - B4SOIVdseff*B4SOIAbovVgst2Vtm ;
T0 = 1.0 - eta;
T1 = 1.0 + eta;
T2 = T1 + 2.0*B4SOIAbulk*B4SOIvtm/(B4SOIVgsteff+1.0e-10);
Lvsat = Leff*(1.0 + B4SOIVdseff /EsatL) ;
T6 = Leff / Lvsat ;
gamma = T6*(0.5*T1 + T0*T0/(6.0*T2));

T3 = T2*T2;
T4 = T0*T0;
T5 = T3*T3;
delta = ((T1/T3)-(5.0*T1 + T2)*T4/(15.0*T5) + T4*T4/(9.0*T5*T2))/(6.0*T6*T6);
T7 = T0/T2;
epsilon = (T7 + T7*T7*T7/3.0)/(6.0*T6);

T8 = B4SOIVgsteff / (EsatL);
T8 = T8 * T8;
npart_c = RNOIC * (1.0 + T8 * TNOIC * Leff);
ctnoi = epsilon / sqrt( gamma * delta) * (2.5316 * npart_c);

if (ctnoi > 1)
    ctnoi=1;
if (ctnoi < 0)
    ctnoi=0;
npart_beta = RNOIA * (1.0 + T8 * TNOIA * Leff);
npart_theta = RNOIB * (1.0 + T8 * TNOIB * Leff);
gamma = gamma * (3.0 * npart_beta * npart_beta);
delta = delta * (3.75 * npart_theta * npart_theta);

B4SOInoiGd0 = NF * beta * B4SOIVgsteff / (1.0 + gche * Rds);
GammaGd0 = gamma * B4SOInoiGd0;
sid = fourkt * GammaGd0;
C0 = NF * B4SOIcox * pParam_B4SOIweffCV * pParam_B4SOIleffCV;
//sigrat = C0 / B4SOInoiGd0 * sqrt(delta / gamma);
sf = (B4SOInoiGd0+1e-15)/sqrt(delta/gamma);

I(di,si) <+ white_noise(MULT_I *sid*abs(1.0-ctnoi * ctnoi), "id");
I(N) <+ MULT_I *V(N) * sf * SCALEN;
I(N) <+ white_noise(MULT_I *sid/{sf*sf*SCALEN*SCALEN}, "cor1");

I(di,si) <+ MULT_I *ctnoi * V(N)*sf*SCALEN ;
I(g1,si) <+ ddt(MULT_Q * 0.5 * C0 * SCALEN * V(N));
I(g1,di) <+ ddt(MULT_Q * 0.5 * C0 * SCALEN * V(N));

#else

$strobe(["BSIMSOI] Although the model selector TNOIMOD is set to 3, the new correla

#endif
end
```

```

3: begin

    //Correlated Thermal Noise by Navid, July 2013

    eta  = 1.0 - B4SOIVdseff*B4SOIAbovVgst2Vtm ;
    T0   = 1.0 - eta;
    T1   = 1.0 + eta;
    T2   = T1 + 2.0*B4SOIAbulk*B4SOIvtm/(B4SOIVgsteff+1.0e-10);
    Lvsat = Leff*(1.0 + B4SOIVdseff / EsatL) ;
    T6    = Leff / Lvsat ;
    gamma = T6*(0.5*T1 + T0*T0/(6.0*T2));

    T3    = T2*T2;
    T4    = T0*T0;
    T5    = T3*T3;
    delta = ((T1/T3)-(5.0*T1 + T2)*T4/(15.0*T5) + T4*T4/(9.0*T5*T2))/(6.0*T6*T6*T6);
    T7    = T0/T2;
    epsilon = (T7 + T7*T7*T7/3.0)/(6.0*T6);

    T8    = B4SOIVgsteff / (EsatL);
    T8    = T8 * T8;
    npart_c = RNOIC * (1.0 + T8 * TNOIC * Leff);
    ctnoi   = epsilon / sqrt( gamma * delta) * (2.5316 * npart_c);

    if (ctnoi > 1)
        ctnoi=1;
    if (ctnoi < 0)
        ctnoi=0;
    npart_beta = RNOIA * (1.0 + T8 * TNOIA * Leff);
    npart_theta = RNOIB * (1.0 + T8 * TNOIB * Leff);
    gamma      = gamma * (3.0 * npart_beta * npart_beta);
    delta      = delta * (3.75 * npart_theta * npart_theta);

    B4SOInoiGd0 = NF * beta * B4SOIVgsteff / (1.0 + gche * Rds);
    GammaGd0    = gamma * B4SOInoiGd0;
    sid         = fourkt * GammaGd0;
    C0          = NF * B4SOIcox * pParam_B4SOIweffCV * pParam_B4SOIleffCV;
    //sigrat     = C0 / B4SOInoiGd0 * sqrt(delta / gamma);
    sf = (B4SOInoiGd0+1e-15)/sqrt(delta/gamma);
    I(di,si) <+ white_noise(MULT_I *sid*abs(1.0-ctnoi * ctnoi), "id");
    I(N)    <+ MULT_I *V(N) * sf * SCALEN;
    I(N)    <+ white_noise(MULT_I *sid/(sf*sf*SCALEN*SCALEN), "cor1");

    I(di,si) <+ MULT_I *ctnoi * V(N)*sf*SCALEN ;
    I(gi,si) <+ ddt(MULT_Q * 0.5 * C0 * SCALEN * V(N));
    I(gi,di) <+ ddt(MULT_Q * 0.5 * C0 * SCALEN * V(N));

end
endcase
if (TNOIMOD != 3) begin
    I(N) <+ V(N);
end
end

```

Description of Bugs:

1. **2018 bug.1:** Stress effect in vfbzb – Coding issue : stress effect is added and updated variable is here_B4SOIvth0, so replacing pParam_B4SOIvth0 by here_B4SOIvth0 (ADI)

```
here_B4SOIvth0 = pParam_B4SOIvth0 + dvth0_lod;  
tmp3 = B4SOItype * pParam_B4SOIvth0 - T2 - T3 + pParam_B4SOIk3 * T4 + T5;
```

Fix: Replace pParam_B4SOIvth0 → here_B4SOIvth0

```
tmp3 = B4SOItype * here_B4SOIvth0 - T2 - T3 + pParam_B4SOIk3 * T4 + T5;
```

tmp3 is further used for calculation of vfbzb.

2. **2018 bug.2:** Self heating module – Updating K1ox and K2ox (ADI)
Issue:

Variable K1ox depends on parameter k1eff,
Variable k2ox depends on model parameter
k2

- ❑ After self-heating module execution k1eff and k2 are updated but k1ox and k2ox are not!

```
pParam_B4SOIk1ox = pParam_B4SOIk1eff * TOX / TOXM;  
here_B4SOIk2ox = here_B4SOIk2 * TOX / TOXM;
```

- Recomputed k1eff and k2 with self heating.

```
here_B4SOIk2=here_B4SOIk2-pParam_B4SOIk2 + T3;  
pParam_B4SOIk1eff = pParam_B4SOIk1 * (1 + pParam_B4SOIk1w1/T0);
```

- Required to update pParam_B4SOIk1ox and here_B4SOIk2ox also

Fix: Defining new instance specific variable: here_B4SOIk1eff

```
here_B4SOIk1eff = pParam_B4SOIk1 * (1 + pParam_B4SOIk1w1/T0);  
pParam_B4SOIk1ox = here_B4SOIk1eff * TOX / TOXM;  
here_B4SOIk2ox = here_B4SOIk2 * TOX / TOXM;
```

Updated k1ox value
Updated k2ox value

3. **2018 bug.3:** BSIMSOI model fitting discrepancy at negative VE (Infineon)
No VA code update required

4. **2018 bug.4:** Nonphysical behavior of Sid and Sig in TNOIMOD=1 (GF)

Fix: Due to unphysical behavior of TNOIMOD=1 for thermal noise, the following warning for TNOIMOD = 1 was added in BSIMSOI technical manual:

```
Warning: TNOIMOD = 1 is not supported and may be removed from future version.
```

5. **2019 bug.1:** Convergence issue at low temperature (CADENCE)

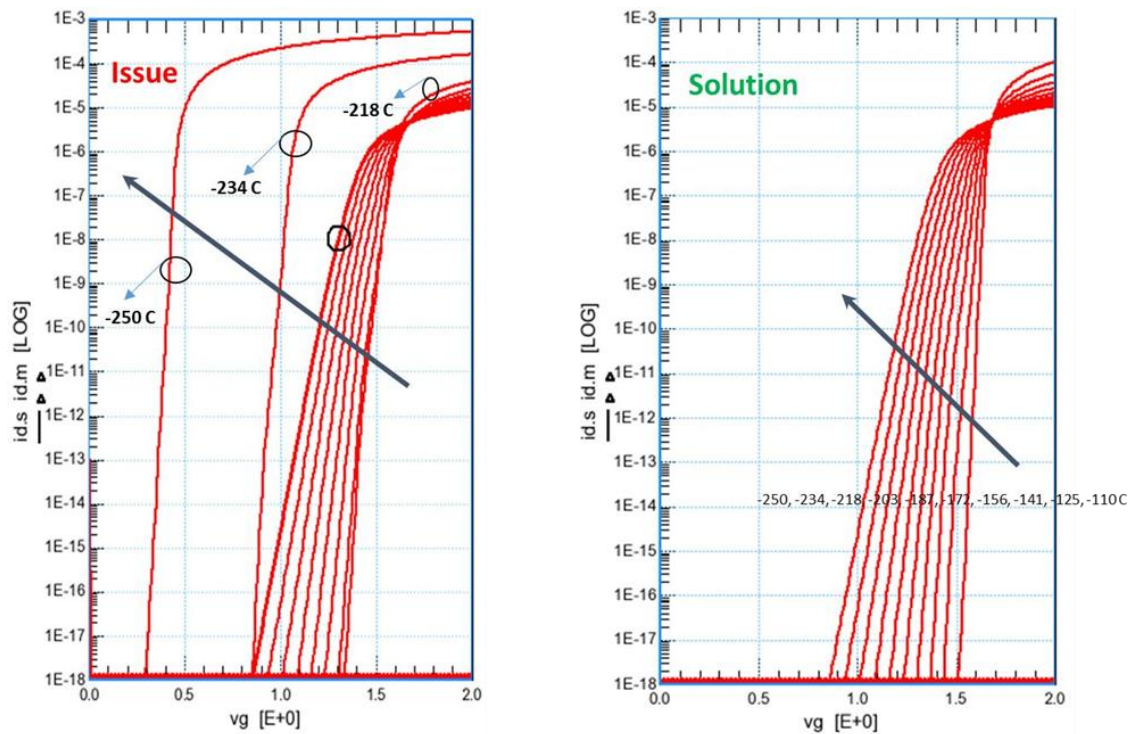
Issue:

- At extremely low temperature n_i could underflow and becomes numerically zero and create division by zero.

- Model has convergence issue for low temperature. Cadence has solved the convergence issue by clamping intrinsic concentration (ni) to fixed value of 1e-130.

Fix: Use logarithmic ni (intrinsic concentration), as follows:

$$\ln_ni = \ln(1.45e10 * (DevTemp / 300.15) * \sqrt{DevTemp / 300.15})) + 21.5565981 - Eg / (2.0 * B4SOIvtm);$$



6. 2019 bug.2: QA test Idvd_diode_st., value 0.25 appears twice in biasList (GF)

Fix: Repetitive bias value is corrected, 0.25 is replaced by 0.15

7. 2020 bug.1: AbulkCVfactor correction – pow (0,0) (ADI)

Issue:

```
`MPRnb( CLC ,0.1e-7 , "m" , "Vdsat parameter for C-V model" )
`MPRnb( CLE ,0.0 , "" , "Vdsat parameter for C-V model" )
```

$$abulkCVfactor = 1.0 + \text{pow}((clc / leff), cle);$$

Pow (0,0) =?

Simulator decides pow(0,CLE) = 0,
regardless of CLE

Simulator decides pow(CLC,0) = 1,
regardless of CLC

Correct
implementation

Fix: Code is updated as follows:

```
if (pParam_b4SOIcle == 0) begin
    pParam_B4SOIabulkCVfactor = 2.0;
end else begin
    pParam_B4SOIabulkCVfactor = 1.0 + pow((pParam_B4SOIclc / pParam_B4SOIleff),
    pParam_B4SOIcle);
end
```

- Also bound CLC with **`MPRcz**

8. 2020 bug.2: Diode leakage components & Negative binning values (CADENCE, GF)

Issue: Issue with diffusion – recombination – tunnelling current implementation due to negative value of binning parameters.

1. The backward injection current in the B-S/D diode can be expressed as

$$I_{bs1} = W_{dios} T_{si} j_{difs} \left(\exp\left(\frac{V_{bs}}{n_{diodes} V_t}\right) - 1 \right)$$

$$I_{bd1} = W_{diod} T_{si} j_{difd} \left(\exp\left(\frac{V_{bd}}{n_{diodes} V_t}\right) - 1 \right)$$

2. The carrier recombination current

$$I_{br2} = W_{dios} T_{si} j_{recs} \left(\exp\left(\frac{V_{bs}}{0.026 n_{recs}}\right) - \exp\left(\frac{V_{sb}}{0.026 n_{recs} V_{rec0s} + V_{sb}}\right) \right)$$

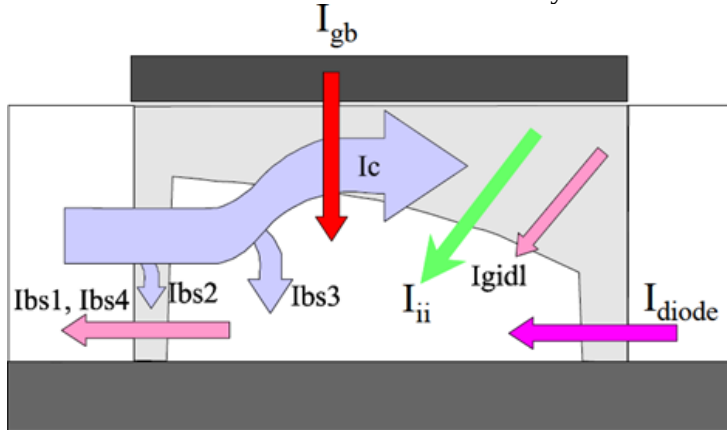
$$I_{bd2} = W_{diod} T_{si} j_{recd} \left(\exp\left(\frac{V_{bd}}{0.026 n_{recd}}\right) - \exp\left(\frac{V_{db}}{0.026 n_{recd} V_{rec0d} + V_{db}}\right) \right)$$

3. The reverse bias tunnelling current

$$I_{bs4} = W_{dios} T_{si} j_{tuns} \left(1 - \exp\left(\frac{V_{sb}}{0.026 n_{tuns} V_{tun0s} + V_{sb}}\right) \right)$$

$$I_{bd4} = W_{diod} T_{si} j_{tund} \left(1 - \exp\left(\frac{V_{db}}{0.026 n_{tund} V_{tun0d} + V_{db}}\right) \right)$$

4. Similarly, for Ibs/d3 current due to recombination in natural body



In the current implementation, J_{difs} and J_{difd} are not limited to negative value; results in unphysical(negative) diffusion current with negative binning parameter values.

Binning:

$$I_{isdif} = I_{SDIF} + I_{ISDIF} * Inv_L + W_{ISDIF} * Inv_W + P_{ISDIF} * Inv_LW;$$

$$T1 = \text{pow}(W_{drn}, L_{WN});$$

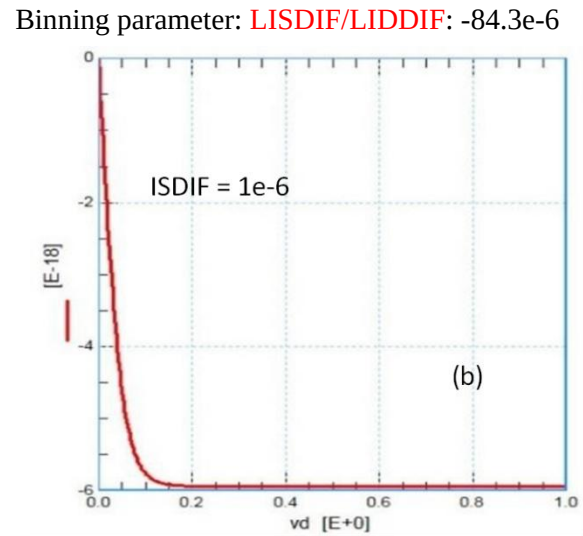
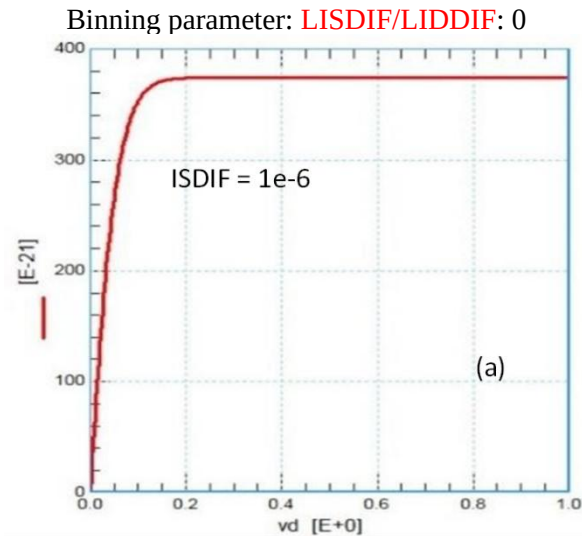
$$j_{difs} = I_{isdif} * T1;$$


```

/* Ibs1: diffusion current */
if (jdifs == 0) begin
    Ibs1 = 0;
end else begin
    T0 = WsTsi * jdifs;
    Ibs1 = T0 * (ExpVbsNVtm - 1);
end

/* Ibd1: diffusion current */
if (jdifd == 0) begin
    Ibd1 = 0;
end else begin
    T0 = WdTsi * jdifd;
    Ibd1 = T0 * (ExpVbdNVtm - 1);
end

```



Fix: Limit the diffusion current from being negative or clamp it to small positive value.

```
if (jdifs<=0) begin
```

The same solution also applies to [recombination current](#), [tunnelling current](#) and [current due to recombination in natural body](#).

9. 2020 bug.3: Operating variable: Correction in Macro definition (ADI)

Issue: Voltages should not be scaled with **\$mfactor >1**

```

// OPERATING POINT VARIABLES
`OPM( VDSAT      , "V"      , "Drain-Source saturation Voltage")
`OPM( VTH        , "V"      , "Threshold Voltage")

```

Fix: Changed **`OPM** to **`OPP** in macro definition.

```

// OPERATING POINT VARIABLES
`OPP( VDSAT      , "V"      , "Drain-Source saturation Voltage")
`OPP( VTH        , "V"      , "Threshold Voltage")

```

10. 2020 bug.4: Parameter's unit update (ADI)

Fix: Following units have been updated (updated units are shown in red colour, text in black represents old code and in green represents an updated code):

'MPRcz(TBGASUB	,7.02e-4	," "	,"First parameter of band-gap change due to temperature")
'MPRcz(TBGBSUB	,1108.0	," "	,"Second parameter of band-gap change due to temperature")
'MPRcz(TBGASUB	,7.02e-4	,"eV/K"	,"First parameter of band-gap change due to temperature")
'MPRcz(TBGBSUB	,1108.0	,"K"	,"Second parameter of band-gap change due to temperature")
'MPRnb(VBX	,0.0	," "	,"Vth transition body Voltage")
'MPRnb(VBX	,0.0	,"V"	,"Vth transition body Voltage")
'MPRnb(KT1L	,0.0	," "	,"Temperature coefficient of Vth")
'MPRnb(KT1L	,0.0	,"V-um"	,"Temperature coefficient of Vth")
Since LEFFEOT has units "um", KT1L should have units "V-um" per this equation: T1 = (pParam_B4SOIkt1 + pParam_B4SOIkt1 / LEFFEOT);			
'MPRnb(UA	,2.25e-9	," "	,"Linear gate dependence of mobility")
'MPRnb(UB1	,-7.61e-18	," "	,"Temperature coefficient of ub")
'MPRnb(UA	,2.25e-9	,"m/V"	,"Linear gate dependence of mobility")
'MPRnb(UB1	,-7.61e-18	,"(m/V)^2"	,"Temperature coefficient of ub")

11. 2020 bug.5: Bug in RDSMOD implementation (ADI)

Issue: There is no “else” for RDSMOD=1. So, the loop for RDSMOD=2 will consider RDSMOD=0 and 2. RDSMOD=0 calculated twice.

Fix: Added else-if for RDSMOD=1.

```

/* v4.0 Calculate bias-dependent external S/D resistance */
if (RDSMOD == 0) begin
  if ((B4SOIsourceResistance + RSC) > MINR) begin
    Rs = B4SOIsourceResistance + RSC_t ;
    if (Rs < MINR) begin
      Rs = MINR;
    end
  end else begin
    Rs = 0;
  end
  if ((B4SOIdrainResistance + RDC) > MINR) begin
    Rd = B4SOIdrainResistance + RDC_t;
    if (Rd < MINR) begin
      Rd = MINR;
    end
  end else begin
    Rd = 0;
  end
end
if (RDSMOD == 1) begin /* 2013.enh5, 2015.enh1
  .
  .
  .
end else begin /* RDMOD == 2
  Rs = 0;
  Rd = 0;
end

```

Old

```

/* v4.0 Calculate bias-dependent external S/D resistance
*/
if (RDSMOD == 0) begin
  if ((B4SOIsourceResistance + RSC) > MINR) begin
    Rs = B4SOIsourceResistance + RSC_t ;
    if (Rs < MINR) begin
      Rs = MINR;
    end
  end else begin
    Rs = 0;
  end
  if ((B4SOIdrainResistance + RDC) > MINR) begin
    Rd = B4SOIdrainResistance + RDC_t;
    if (Rd < MINR) begin
      Rd = MINR;
    end
  end else begin
    Rd = 0;
  end
end
else if (RDSMOD == 1) begin /* 2013.enh5,
2015.enh1
  .
  .
  .
end else begin /* RDMOD == 2
  Rs = 0;
  Rd = 0;
end

```

New

12. 2020 bug.6: Technical manual update: Tnom correction (UCB)

Issue: TNOM is a user parameter in degree Celsius, and inside the BSIM codes, we convert it to variable Tnom = TNOM + `P_CELSIUS0 as all the temperature calculations use Kelvin temperature.

In the following equation of technical manual **TNOM** should be **Tnom**:

$$V_{bci} = VBCI \left(1 + TVBCI \left(\frac{T}{TNOM} - 1 \right) \right)$$

Fix: “TNOM” is replaced with “Tnom” in the temperature expression in the manual.

13. 2021 bug.1: Negative value of CDD (CADENCE)

Issue: In BSIM-SOI v4.7.0_Beta3, CDD goes negative when Vds is negative.

Fix: BSIM-SOI v4.7.0_Beta4 fixes this bug by taking $CLE \geq 0.04$. A comment is introduced in the manual for the users to use a value for CLE greater than or equal to 0.04 to address this issue

14. 2021 bug.2: vampyre warning “possible division by zero for expression 1/(1-ASD)” (UCB)

Issue: In BSIM-SOI v4.7.0 Beta3 when ASD=1, 1/(1-ASD) gives 1/0 error.

Fix: BSIM-SOI v4.7.0 Beta4 fixes this bug by changing the limits for ASD - (0, 1) limit is used for ASD.

SIM-SOI v4.7.0 Beta3:

```
`MPRoz( ASD, 0.3, "", "Source/drain bottom diffusion smoothing parameter" )
```

BSIM-SOI v4.7.0 Beta4:

```
`MPRoz( ASD, 0.3, "", 0, 1, "Source/drain bottom diffusion smoothing parameter" )
```

Also the following fatal error strobe is removed from the code since this becomes redundant after limiting ASD:

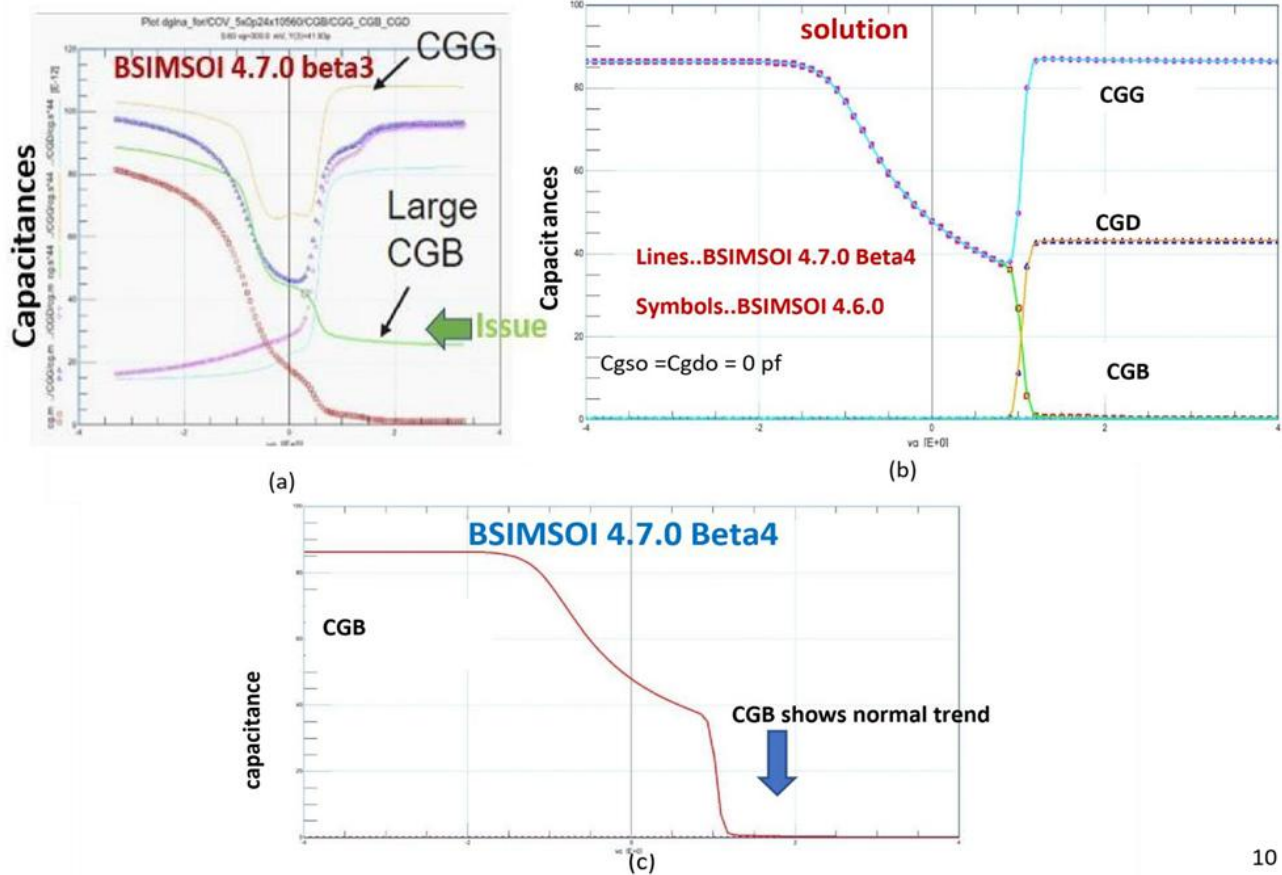
```
if (B4SOIcsdmin > 0 && (ASD <= 0.0 || ASD >= 1.0)) begin
    $strobe("Fatal: Asd = %g should be within (0, 1).", ASD);
    Fatal_Flag = 1;
end
```

15. 2021 bug.3: Incorrect value of CGB (UCB)

Issue: In BSIM-SOI v4.7.0 Beta3 release, assignment of charges to nodes was done wrongly, due to this CGB was showing high value.

Fix: The correct implementation is shown below:

Implementation in BSIMSOI 4.7.0 Beta3	Updated Implementation in upcoming beta4 code:
<pre>// [Nov 2016]: adding overlap charges (qgso + qgdo) to qgate// [Qov depends on V(gm,si) for RGATEMOD=3, else on V(gi,si)] qgi = qgate; qov = qgso + qgdo; qgate = qgi + qov;</pre>	<pre>// [Nov 2016]: adding overlap charges (qgso + qgdo) to qgate// [Qov depends on V(gm,si) for RGATEMOD=3, else on V(gi,si)] qgi = qgate; qdi = qdrn; qsi = qsrc; qov = qgso + qgdo; qdrn = qdi - qgdo; qsrc = qsi - qgso; qgate = qgi + qov;</pre> ← Corrected Contribution of Overlap Charges



(a) In BSIMSOI4.7.0 Beta3 CGB obtained was high due to wrong assignment of node charges. Problem is solved in Beta4.(b) BSIMSOI 4.6.0 & BSIMSOI 4.7.0 Beta4 are compared.(c) CGB in BSIMSOI 4.7.0 Beta4 shows normal trend as expected.

10

16. 2022 bug.1: Allow minr=0 (ADI)

Issue: In BSIM-SOI v4.7.0 Beta3 release, assignment of charges to nodes was done wrongly, due to this CGB was showing high value parameter "minr" uses MPROZ() macro which excludes zero.

Fix: MPRcz() macro is used that includes zero.

Implementation:

Beta4 code:

```
MPROZ( MINR, $simparam("minr", 1.0e-3), "ohm", "Minimum S/D resistance")
```

Updated implementation in upcoming release:

```
MPRcz( MINR, $simparam("minr", 1.0e-3), "ohm", "Minimum S/D resistance")
```

17. 2023 bug.1: Revert IDS0MULT parameter (ADI)

Issue: IDS0MULT parameter was deleted in BSIM-SOI v4.7.0 Beta5 after MULT implementation, later it was suggested to keep both MULT and IDS0MULT parameter.

Fix: The IDS0MULT parameter is added again. The updated code contains both MULT and IDS0MULT parameters.

```
`MPRoo( IDSOMULT ,1.0 ,"" ,0 ,inf
`MPRoo( UOMULT ,1.0 ,"" ,0 ,inf
```

18. 2023 bug.2: Improved MULT implementation (GF)

Fix: Conductance's used in white noise are scaled as follows:

```
gdnoise = MULT_I / Rd ;
gsnoise = MULT_I / Rs ;
```

```
if (RDSMOD != 2 && (B4SOIdrainResistance + RDC) >= MINR) begin
    I(d, di) <+ MULT_I * V(d,di) / Rd ;
    I(d, di) <+ white_noise( abs(MULT_I * fourkt * gdnoise), "rd");
end else begin
    V(d, di) <+ 0.0;
end
if (RDSMOD != 2 && (B4SOISourceResistance + RSC) >= MINR) begin
    I(s, si) <+ MULT_I * V(s,si) / Rs ;
    I(s, si) <+ white_noise( abs(MULT_I * fourkt * gsnoise), "rs");
end else begin
    V(s, si) <+ 0.0;
end
```

19. 2023 bug.3: Updating fundamental constants to pass QA tests (UCB)

Issue: Different simulators uses different values of constants.

Fix:

1. NIST Constant used `P_K_NIST1998, P_Q_NIST1998
2.

```
if ($ModelSelect =~ /^bsimimg·||·bsimsoi·/i) {
    print OF "+ minr=1e-3";
}
```
3. Updates for tests (for nmos) 83, 87, 91, 92, 95 & 96: Model parameters AGB1=3.7622e-8

20. 2024 bug.1: The range of FDMOD is "no bound"; should be restricted to 0 or 1 (ADI)

Fix: MPIcc() macro is used that sets bound on FDMOD

BSIM-SOI v4.7.0 Beta6:

```
`MPInb( FDMOD ,0 ,"" ,0 ,1 , "Improved dVbi model selector" )
```

BSIM-SOI v4.7.0 Beta7:

```
`MPIcc( FDMOD ,0 ,"" ,0 ,1 , "Improved dVbi model selector" )
```


21. 2024 bug.2: Possible division by zero for few parameters (ADI)

Fix:

Current Implementation in BSIM-SOI 4.7.0 beta6:

```
`MPRco( UOMULT ,1.0 ,"" ,0 ,1 , "Variability in carrier mobility" )
`MPRcz( EOT ,100.0e-10 , "m" , "Effective SiO2 thickness" )
`MPRnb( TEMPEOT ,300.15 , "K" , "Temperature for extraction of EOT" )
`MPRnb( VTM00 ,0.026 , "V" , "Hard coded 25 degC thermal voltage" )
```

Updated Implementation in BSIM-SOI 4.7.0 beta6:

```
`MPRco( UOMULT ,1.0 ,"" ,0 ,inf , "Variability in carrier mobility" )
`MPRco( EOT ,100.0e-10 , "m" ,0.0 ,inf , "Effective SiO2 thickness" )
`MPRoz( TEMPEOT ,300.15 , "K" , "Temperature for extraction of EOT" )
`MPRoz( VTM00 ,0.026 , "V" , "Hard coded 25 degC thermal voltage" )
```

22. 2024 bug.5: Correct RDSMOD=1 implementation in code (UCB/IITK)

Issue: PRWB was used for “Rs(V)” calculation instead of pParam_B4SOIprwe.

Fix: In the updated code (BSIM-SOI v4.7.0 Beta8) we have corrected it to pParam_B4SOIprwe.

Implementation in BSIM-SOI 4.7.0 beta7

```
end else if (RDSMOD == 1) begin /* 2013.enh5, 2015.enh1 */ /* Updated in v4.6 */
  Rds = 0.0;
  /* Rs(V) */
  T0 = vgs - pParam_B4SOIvfbssd;
  T1 = sqrt(T0 * T0 + 1.0e-4);
  vgs_eff = 0.5 * (T0 + T1);

  T0 = 1.0 + pParam_B4SOIprwg * vgs_eff;
  T1 = -pParam_B4SOIprwb * vbs;
  T2 = 1.0 / T0 + T1 + PRWB * (ves - vfbb);
  T3 = T2 + sqrt(T2 * T2 + 0.01);

  T4 = rs0 * 0.5;
  Rs = rswmin + T3 * T4 + B4SOIsourceResistance + RSC_t;
  if (Rs < MINR) begin
    Rs = MINR;
  end

  /* Rd(V) */
  T0 = vgd - pParam_B4SOIvfbssd;
  T1 = sqrt(T0 * T0 + 1.0e-4);
  vgd_eff = 0.5 * (T0 + T1);

  T0 = 1.0 + pParam_B4SOIprwg * vgd_eff;
  T1 = -pParam_B4SOIprwb * vbd;
  T2 = 1.0 / T0 + T1 + pParam_B4SOIprwe * (ves - vfbb);
  T3 = T2 + sqrt(T2 * T2 + 0.01);
  T4 = rd0 * 0.5;
  Rd = rdwmin + T3 * T4 + B4SOIdrainResistance + RDC_t;
  if (Rd < MINR) begin
    Rd = MINR;
  end
end
```

Implementation in BSIM-SOI 4.7.0 beta8

```

end else if (RDSMOD == 1) begin /* 2013.enh5, 2015.enh1 */ /* Updated in v4.6 */
  Rds = 0.0;
  /* Rs(V) */
  T0 = vgs - pParam_B4SOIvfbds;
  T1 = sqrt(T0 * T0 + 1.0e-4);
  vgs_eff = 0.5 * (T0 + T1);

  T0 = 1.0 + pParam_B4SOIprwg * vgs_eff;
  T1 = -pParam_B4SOIprwb * vbs;
  T2 = 1.0 / T0 + T1 + pParam_B4SOIprwe * (ves - vfbb);
  T3 = T2 + sqrt(T2 * T2 + 0.01);

  T4 = rs0 * 0.5;
  Rs = rswmin + T3 * T4 + B4SOIsourceResistance + RSC_t;
  if (Rs < MINR) begin
    Rs = MINR;
  end

  /* Rd(V) */
  T0 = vgd - pParam_B4SOIvfbds;
  T1 = sqrt(T0 * T0 + 1.0e-4);
  vgd_eff = 0.5 * (T0 + T1);

  T0 = 1.0 + pParam_B4SOIprwg * vgd_eff;
  T1 = -pParam_B4SOIprwb * vbd;
  T2 = 1.0 / T0 + T1 + pParam_B4SOIprwe * (ves - vfbb);
  T3 = T2 + sqrt(T2 * T2 + 0.01);
  T4 = rd0 * 0.5;
  Rd = rdwmin + T3 * T4 + B4SOIdrainResistance + RDC_t;
  if (Rd < MINR) begin
    Rd = MINR;
  end
end

```

23. 2024 bug.6: Correct source/drain resistance equations for RDSMOD = 1 in technical manual ([UCB/IITK](#))

Issue: Rs(V) and Rd(V) equations in the code have substrate to source voltage (ves), however, in the manual, it was inaccurately typed as internal body to gate-source voltage (vbGS).

Fix: In the updated manual (BSIM-SOI v4.7.0 Beta8) we have corrected it to **ves** from **vbGS**.

BSIM-SOI v4.7.0 Beta7 Manual:

- $rdsMod = 1$ (External $R_d(V)$ and $R_s(V)$)

$$R_s(V) = \frac{RSWMIN + RSW \cdot \left[-PRWB \cdot V_{bs} - PRWE(V_{bGS} - V_{FBb}) + \frac{1}{1 + PRWG \cdot (V_{gs} - V_{fbsd})} \right]}{(1e6 \cdot W_{eff})^{WR} \cdot NF}$$

$$R_d(V) = \frac{RDWMIN + RDW \cdot \left[-PRWB \cdot V_{bd} - PRWE(V_{bGS} - V_{FBb}) + \frac{1}{1 + PRWG \cdot (V_{gd} - V_{fbsd})} \right]}{(1e6 \cdot W_{eff})^{WR} \cdot NF}$$

BSIM-SOI v4.7.0 Beta8 Manual:

- $rdsMod = 1$ (External $R_d(V)$ and $R_s(V)$)

$$R_s(V) = \frac{RSWMIN + RSW \cdot \left[-PRWB \cdot V_{bs} - PRWE(V_{es} - V_{FBb}) + \frac{1}{1 + PRWG \cdot (V_{gs} - V_{fbsd})} \right]}{(1e6 \cdot W_{eff})^{WR} \cdot NF}$$

$$R_d(V) = \frac{RDWMIN + RDW \cdot \left[-PRWB \cdot V_{bd} - PRWE(V_{es} - V_{FBb}) + \frac{1}{1 + PRWG \cdot (V_{gd} - V_{fbsd})} \right]}{(1e6 \cdot W_{eff})^{WR} \cdot NF}$$

24. 2024 bug.7: Issues with TRm1 replacing (TempRatio -1) (CADENCE)

BSIM-SOI v4.7.0 Beta8:

```
T4 = `Eg300 / B4SOIvtm * TRm1;
/* source side */
T7 = pParam_B4SOIxbjt * T4 / pParam_B4SOIndiode;
`DEXP(T7, TRm1)
T7 = pParam_B4SOIxdif * T4 / pParam_B4SOIndiode;
`DEXP(T7, T1)
T7 = pParam_B4SOIxrec * T4 / pParam_B4SOInrecf0;
`DEXP(T7, T2)
pParam_B4SOIahli0s = pParam_B4SOIahli * TRm1;
pParam_B4SOIjbjts = pParam_B4SOIisbjt * TRm1;
pParam_B4SOIjdifs = pParam_B4SOIisdif * T1;
pParam_B4SOIjrecs = pParam_B4SOIisrec * T2;
T7 = pParam_B4SOIxtun * TRm1;
`DEXP(T7, TRm1)
pParam_B4SOIjtuns = pParam_B4SOIistun * TRm1;

/* drain side */
T7 = pParam_B4SOIxbjt * T4 / pParam_B4SOIndioded;
`DEXP(T7, TRm1)
T7 = pParam_B4SOIxdifd * T4 / pParam_B4SOIndioded;
`DEXP(T7, T1)
T7 = pParam_B4SOIxrecd * T4 / pParam_B4SOInrecf0d;
`DEXP(T7, T2)

pParam_B4SOIahli0d = pParam_B4SOIahlid * TRm1;
pParam_B4SOIjbjtd = pParam_B4SOIidbjt * TRm1;
pParam_B4SOIjdifd = pParam_B4SOIiddif * T1;
pParam_B4SOIjrecd = pParam_B4SOIidrec * T2;
T7 = pParam_B4SOIxtund * TRm1;
`DEXP(T7, TRm1)
pParam_B4SOIjtund = pParam_B4SOIidtun * TRm1;
```

BSIM-SOI v4.7.0 Beta9:

```
T4 = `Eg300 / B4SOIvtm * TRm1;
/* source side */
T7 = pParam_B4SOIxbjt * T4 / pParam_B4SOIndiode;
`DEXP(T7, T0)
T7 = pParam_B4SOIxdif * T4 / pParam_B4SOIndiode;
`DEXP(T7, T1)
T7 = pParam_B4SOIxrec * T4 / pParam_B4SOInrecf0;
`DEXP(T7, T2)
pParam_B4SOIahli0s = pParam_B4SOIahli * T0;
pParam_B4SOIjbjts = pParam_B4SOIisbjt * T0;
pParam_B4SOIjdifs = pParam_B4SOIisdif * T1;
pParam_B4SOIjrecs = pParam_B4SOIisrec * T2;
T7 = pParam_B4SOIxtun * TRm1;
`DEXP(T7, T0)
pParam_B4SOIjtuns = pParam_B4SOIistun * T0;

/* drain side */
T7 = pParam_B4SOIxbjt * T4 / pParam_B4SOIndioded;
`DEXP(T7, T0)
T7 = pParam_B4SOIxdifd * T4 / pParam_B4SOIndioded;
`DEXP(T7, T1)
T7 = pParam_B4SOIxrecd * T4 / pParam_B4SOInrecf0d;
`DEXP(T7, T2)

pParam_B4SOIahli0d = pParam_B4SOIahlid * T0;
pParam_B4SOIjbjtd = pParam_B4SOIidbjt * T0;
pParam_B4SOIjdifd = pParam_B4SOIiddif * T1;
pParam_B4SOIjrecd = pParam_B4SOIidrec * T2;
T7 = pParam_B4SOIxtund * TRm1;
`DEXP(T7, T0)
pParam_B4SOIjtund = pParam_B4SOIidtun * T0;
```

25. 2024 bug.8: Code cleaning – B4SOItnom is replaced with Tnom, and (Tratio -1) is replaced with TRm1 (UCB/IITK)

Fix:

- The variables **B4SOItnom** and **Tratiominus1** have been removed.
- Everywhere in the code, **(Tratio – 1)** has been replaced with **TRm1**.
- **B4SOItnom** replaced with **Tnom**.

26. 2025 bug.1: Vampyre Warning “possible division by zero for parameter “SCALEN” (UCB/IITK)

Fix: BSIM-SOI_4.7.0_Beta10 fixes this by changing the macro definition to “MProz” which excludes zero from the parameter range limit.

27. 2025 bug.2: VA path correction in qaSpec file and result folder path correction in QA Makefile (ADI)

Fix:

1. The path to the Verilog-A file was changed from `../code/bsimsoi.va` to `../bsimsoi.va` in BSIMSOI 4.7.0 Beta10. The path is now corrected back to “`../code/bsimsoi.va`” in BSIMSOI 4.7.0 Beta11.
2. The path for qaSpec file and results folder was changed from `qaSpec` to `/qaSpec` and `results` to `/results`. These paths are now corrected as follows:

```
qaSpecFile = qaSpec
qaResultsDirectory = results
```